

CSE420 Compiler Design

Topics Covered

1. Introduction to Intermediate Code Generation
2. Array Access Logic and SDT for Array Accesses
3. Handling Flow-of-Control Statements

Prepared by Azmari Sultana

Contents

1	Introduction to Intermediate Code Generation	2
1.1	Learning Introduction	2
1.2	Intermediate Representations	2
1.3	Syntax Trees and Directed Acyclic Graphs (DAGs)	3
1.4	Three-Address Code	6
1.5	Representing Three-Address Code: Quadruples, Triples, and Indirect Triples	9
1.6	Static Single-Assignment (SSA) Form	15
1.7	Practice Problems	16
2	Array Access Logic and SDT for Array References	18
2.1	Learning Introduction	18
2.2	Prerequisite: Translating Simple Expressions	18
2.3	Addressing Array Elements	19
2.4	Grammar and SDT for Array References	22
2.5	Worked Simulations of Array-Reference Translation	24
2.6	Practice Problems	28
3	Handling Flow-of-Control Statements	33
3.1	Learning Introduction	33
3.2	Boolean Expressions: Two Roles, and a Grammar	33
3.3	Short-Circuit (Jumping) Code	34
3.4	Translating if-, if-else-, and while-Statements	35
3.5	Worked Simulation — Translating an if-Statement with Mixed OR/AND	39
3.6	Avoiding Redundant Gotos: The Fall-Through Technique	40
3.7	Boolean Values and Jumping Code	42
3.8	Full Simulation — A while-Loop with an OR Condition Followed by a Statement .	44
3.9	Practice Problems	47

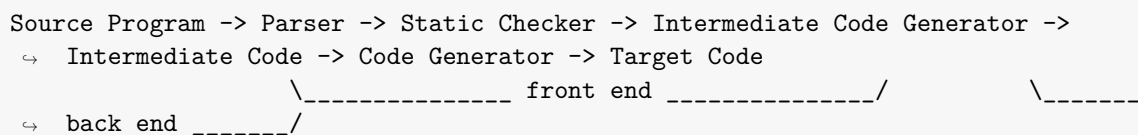
1 Introduction to Intermediate Code Generation

1.1 Learning Introduction

What this topic is. Intermediate code generation is the phase of a compiler that translates a syntactically and semantically checked source program into a machine-independent representation, called *intermediate code*, before final target code is produced.

Why it is needed. A compiler is usually organized so that the front end (lexical analysis, parsing, semantic analysis) is separated from the back end (code optimization and target code generation). If the front end produced target-machine code directly, a compiler for language i and machine j would have to be built as one indivisible unit. By inserting a common intermediate representation in between, we only need m front ends (one per source language) and n back ends (one per target machine) to build all $m \times n$ compilers, instead of building each combination separately. This is the classic *retargetability* argument for intermediate code.

Where it fits in the compilation process. Intermediate code generation sits at the boundary between the front end and the back end:



Parsing, static (type) checking, and intermediate-code generation are frequently done together, in one pass, using syntax-directed translation. Static checking includes type checking, which ensures operators are applied to compatible operands, and other syntactic checks that survive parsing (for example, verifying that a **break** statement in C is properly enclosed in a loop or switch statement).

1.2 Intermediate Representations

A compiler may use a *sequence* of intermediate representations while translating a program:

Source Program $\rightarrow$ High-Level IR $\rightarrow \dots \rightarrow$ Low-Level IR $\rightarrow$ Target Code

- **High-level representations** (e.g., syntax trees) are close to the source language. They depict the natural hierarchical structure of the program and are well suited to tasks such as static type checking.
- **Low-level representations** (e.g., three-address code) are close to the target machine. They are suitable for machine-dependent tasks such as register allocation and instruction selection.

Three-address code can range from high level to low level depending on the choice of operators used. For expressions, the difference between a syntax tree and three-address code is largely superficial. For statements that involve flow of control (loops, conditionals), the difference is more significant: a syntax tree represents only the components of the statement, while three-address code contains explicit labels and jump instructions that represent the flow of control directly, the way machine code does.

An intermediate representation may be an actual language (such as C, which some compilers emit as an intermediate step) or it may be an internal data structure shared among the phases of the compiler.

1.2.1 Exam Revision Questions

Q1. What is intermediate code generation, and why is it used in a compiler?

Answer: Intermediate code generation is the compiler phase that translates a checked source program into a machine-independent intermediate representation. It is used so that a compiler's front end (language-specific analysis) can be kept separate from its back end (machine-specific code generation). This separation allows m front ends and n back ends to be combined to build $m \times n$ compilers, instead of writing a separate translator for every language-machine pair, and it also makes the retargeting of a compiler to a new machine and the addition of machine-independent optimizations much easier.

Q2. Where does intermediate code generation fit into the structure of a compiler front end?

Answer: It comes after parsing and static (type) checking and before the back end's code generator. The typical pipeline is Parser $\rightarrow$ Static Checker $\rightarrow$ Intermediate Code Generator $\rightarrow$ Code Generator. These three front-end phases are often combined and executed together during a single syntax-directed pass over the input.

Q3. Differentiate between a high-level and a low-level intermediate representation.

Answer:

Aspect	High-Level Representation	Low-Level Representation
Closeness	Close to the source language	Close to the target machine
Example	Syntax tree	Three-address code
Structure	Depicts hierarchical structure of the program	Uses linear instructions with explicit labels/jumps
Best suited for	Static type checking	Register allocation, instruction selection, optimization

Q4. Why might a compiler use more than one intermediate representation?

Answer: Different compiler phases need different properties from the code. Early phases (such as type checking) benefit from a high-level, hierarchical representation like a syntax tree, while later phases (such as optimization and code generation) benefit from a low-level, flatter representation like three-address code that is closer to real machine instructions. A compiler may therefore transform the program through a sequence of representations, each suited to the phase that consumes it.

1.3 Syntax Trees and Directed Acyclic Graphs (DAGs)

1.3.1 Syntax Trees versus DAGs

A **syntax tree** depicts the natural hierarchical structure of a source-language construct. Each interior node represents an operator, and its children represent the operands.

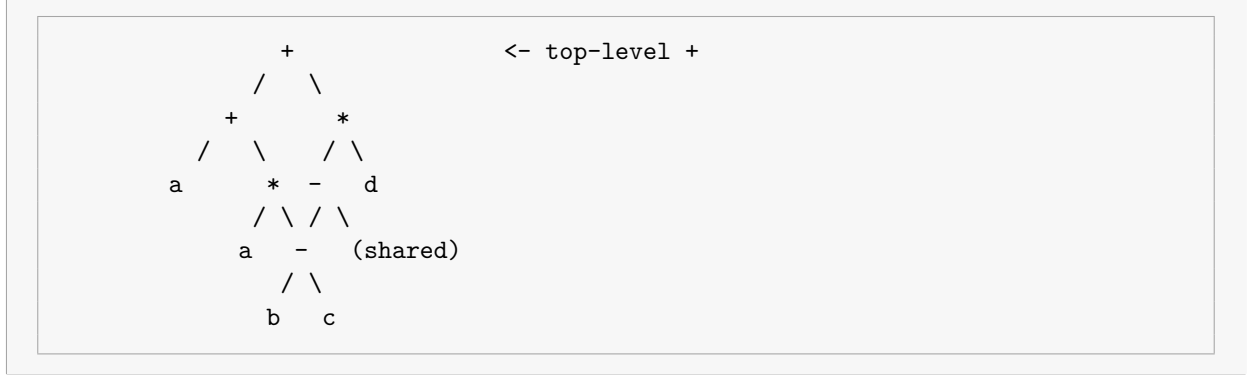
A **directed acyclic graph (DAG)** for an expression is like a syntax tree, but a DAG identifies the *common subexpressions* of the expression — subexpressions that occur more than once. In a syntax tree, a subexpression that occurs twice would be duplicated as two separate subtrees; in a DAG, a node that represents a repeated subexpression has more than one parent, so it is shared rather than repeated. Consequently, a DAG not only represents an expression more compactly, it also tells the compiler that the shared subexpression needs to be computed only once — an important clue for generating efficient code.

Example 1 (DAG construction). Consider the expression

$$a + a * (b - c) + (b - c) * d$$

- The identifier **a** appears twice, so its leaf has two parents.
- The subexpression **b - c** appears twice (once in **a*(b-c)** and once in **(b-c)*d**), so it is represented by a *single* - node with two parents — this node's value need be computed only once.

The resulting DAG is:



Both occurrences of **b - c** point to the same - node, and both occurrences of **a** point to the same leaf.

1.3.2 SDD for Constructing Syntax Trees or DAGs

The same syntax-directed definition (SDD) can build either a syntax tree or a DAG, depending only on whether the node-construction functions **Leaf** and **Node** create a brand-new node every time, or first check whether an identical node already exists and reuse it.

Production	Semantic Rule
$E \rightarrow E_1 + T$	$E.node = \mathbf{new} \text{ Node}('+', E_1.node, T.node)$
$E \rightarrow E_1 - T$	$E.node = \mathbf{new} \text{ Node}('-', E_1.node, T.node)$
$E \rightarrow T$	$E.node = T.node$
$T \rightarrow (E)$	$T.node = E.node$
$T \rightarrow \mathbf{id}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{id}, id.entry)$
$T \rightarrow \mathbf{num}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{num}, num.val)$

- If **Leaf** and **Node** always allocate a fresh node, the SDD produces a **syntax tree**.
- If, before creating a new node, **Node(op, left, right)** first checks whether a node with that same label **op** and the same children **left, right** already exists, and returns the existing node instead of creating a duplicate, the SDD produces a **DAG**.

Simulation: building the DAG step by step. The table below reproduces the exact sequence of calls that constructs the DAG for $a + a * (b - c) + (b - c) * d$, assuming **Leaf** and **Node** reuse existing nodes wherever possible.

Step	Call	Result	Reused?
p_1	$Leaf(\mathbf{id}, entry-a)$	new leaf for a	No — first a
p_2	$Leaf(\mathbf{id}, entry-a)$	$p_2 = p_1$	Yes — same a

Step	Call	Result	Reused?
p_3	$Leaf(\mathbf{id}, entry-b)$	new leaf for b	No
p_4	$Leaf(\mathbf{id}, entry-c)$	new leaf for c	No
p_5	$Node('-', p_3, p_4)$	node for b - c	No — first b-c
p_6	$Node('*', p_1, p_5)$	node for a*(b-c)	No
p_7	$Node('+', p_1, p_6)$	node for a + a*(b-c)	No
p_8	$Leaf(\mathbf{id}, entry-b)$	$p_8 = p_3$	Yes — same b
p_9	$Leaf(\mathbf{id}, entry-c)$	$p_9 = p_4$	Yes — same c
p_{10}	$Node('-', p_3, p_4)$	$p_{10} = p_5$	Yes — identical signature $\langle -, p_3, p_4 \rangle$ already exists
p_{11}	$Leaf(\mathbf{id}, entry-d)$	new leaf for d	No
p_{12}	$Node('*', p_5, p_{11})$	node for (b-c)*d	No
p_{13}	$Node('+', p_7, p_{12})$	final DAG root	No

The key step is p_{10} : when we ask for $Node('-', \mathbf{b}, \mathbf{c})$ a second time, a node with that exact operator and those exact children already exists (created at p_5), so the existing node is returned rather than a new one being built. This is precisely what turns a tree into a DAG.

1.3.3 The Value-Number Method for Constructing DAGs

In an implementation, the nodes of a syntax tree or DAG are stored as records in an array. Each row of the array is one record — one node. The integer index of a node's record in this array is called the **value number** of that node (or of the expression it represents).

For an interior node, define its **signature** as the triple $\langle op, l, r \rangle$, where op is the node's label and l, r are the value numbers of its left and right children (a unary operator is treated as having $r = 0$).

Algorithm (Value-Number Method for constructing DAG nodes).

- **Input:** a label op , and nodes (value numbers) l and r .
- **Output:** the value number of a node in the array having signature $\langle op, l, r \rangle$.
- **Method:** Search the array for a node M with label op , left child l , and right child r . If such a node exists, return its value number. Otherwise, create a new node N with label op , left child l , right child r , append it to the array, and return its (new) value number.

Searching the whole array on every call is expensive. In practice, a **hash table** is used: nodes are grouped into “buckets” using a hash function $h(op, l, r)$ that spreads signatures evenly across buckets. Each bucket is a linked list of the value numbers whose signature hashes to that bucket. To look up a node, the compiler computes the bucket index $h(op, l, r)$ and scans only the (short) list of value numbers within that bucket, comparing each candidate's actual signature to $\langle op, l, r \rangle$; a match is returned, and if the list is exhausted with no match, a new cell (and, if necessary, a new node) is created and added to that bucket. This makes DAG construction close to constant time per node, independent of the number of nodes already built.

1.3.4 Exam Revision Questions

Q1. What is a directed acyclic graph (DAG) in the context of intermediate code generation, and how does it differ from a syntax tree?

Answer: A DAG for an expression is a compact form of a syntax tree in which every *common subexpression* — a subexpression appearing more than once — is represented by a single shared node with multiple parents, instead of being duplicated. A syntax tree, in contrast, replicates the subtree for every occurrence of a repeated subexpression. The DAG therefore both saves space and tells the compiler that a shared subexpression needs to be evaluated only once.

Q2. How can a single SDD be used to build either a syntax tree or a DAG?

Answer: The same semantic rules (`E.node = new Node(op, ..., ...)`, etc.) are used in both cases. The difference lies only in the implementation of the `Leaf` and `Node` constructor functions: if they always allocate a brand-new node, the result is a syntax tree; if they first check whether a node with the identical label and children already exists and, if so, return that existing node instead of creating a new one, the result is a DAG.

Q3. Define value number. What is a node's signature?

Answer: The value number of a node is the index of its record within the array that stores all the nodes of a syntax tree/DAG. The signature of an interior node is the triple $\langle op, l, r \rangle$, where op is its operator label and l, r are the value numbers of its left and right children (with $r = 0$ for a unary operator).

Q4. Why are hash tables used when constructing DAGs, instead of linearly searching the node array?

Answer: A linear search of the entire node array for every new subexpression is expensive, especially for large programs. A hash table groups nodes into buckets using a hash function of the signature $\langle op, l, r \rangle$, so that only the small number of nodes in one bucket need to be examined, giving close to constant-time lookup and insertion.

Q5. Construct the DAG for the expression $a + a * (b - c) + (b - c) * d$ and identify all shared nodes.

Answer: The DAG has one leaf for **a** (shared by both uses of **a**), one leaf each for **b**, **c**, and **d**, one **-** node for **b - c** (shared by **a*(b-c)** and **(b-c)*d**), one ***** node for **a*(b-c)**, one ***** node for **(b-c)*d**, one **+** node combining **a** and **a*(b-c)**, and a final **+** node combining that result with **(b-c)*d**. The two shared nodes are the leaf for **a** and the **-** node for **b-c**; every other node has exactly one parent.

1.4 Three-Address Code

1.4.1 What Three-Address Code Is

Three-address code (3AC) is a linear sequence of instructions in which each instruction has **at most one operator** in addition to the assignment operator; no built-up (nested) arithmetic expressions are allowed. The general form of an assignment instruction is

$$x = y \text{ op } z$$

where x, y, z are addresses and op is a single operator. The name “three-address” comes from the fact that such an instruction typically mentions three addresses: two for the operands (y, z) and one for the result (x).

A multi-operator source expression such as $x + y * z$ cannot be written as a single three-address instruction, since it has two operators. It is instead unraveled into a sequence of instructions with at most one operator each, using compiler-generated **temporary names** ($t_1, t_2, \dots$) to hold intermediate values:

```
t1 = y * z
t2 = x + t1
```

1.4.2 Why Three-Address Code Is Used

- It exposes the structure of complex expressions and nested flow-of-control statements as a flat sequence of simple steps, which is exactly the granularity needed for target-code generation and machine-independent optimization.
- The use of explicit temporary names for every intermediate value makes it easy for the compiler to **rearrange** three-address code (an important enabler of optimization).
- Three-address code is a linearized representation of a syntax tree or a DAG: each interior node of the tree/DAG becomes one three-address instruction, with the node's operator becoming the instruction's **op**, and the node's temporary name becoming the instruction's result.

Worked Example — from DAG to three-address code. Consider again the DAG for $a + a * (b - c) + (b - c) * d$. Reading it from the leaves upward and giving each interior node its own temporary produces:

```
t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4
```

Because the DAG has only *one* node for $b - c$ (shared), that subtraction is computed only once, into **t1**, and **t1** is reused in both $t2 = a * t1$ and $t4 = t1 * d$. This is exactly why DAGs are useful before three-address code is generated — they reveal which computations can be shared.

1.4.3 Addresses and Instructions

Three-address code is built from two basic concepts: **addresses** and **instructions**.

An address can be one of the following:

- **A name.** Source-program names may appear directly as addresses; in an implementation, a name is replaced by a pointer to its symbol-table entry.
- **A constant.** The compiler must handle the various constant types of the source language.
- **A compiler-generated temporary.** A fresh temporary name is created every time one is needed. This is especially useful in optimizing compilers, where distinct temporaries make it easy to move code around; multiple temporaries can later be merged into the same register.

Common three-address instruction forms:

1. **Assignment:** $x = y \text{ op } z$, where *op* is a binary arithmetic or logical operation.
2. **Unary assignment:** $x = \text{op } y$, where *op* is a unary operation such as unary minus, logical negation, a shift operator, or a type-conversion operator.
3. **Copy instruction:** $x = y$, which assigns the value of *y* to *x*.
4. **Unconditional jump:** **goto** *L*. The three-address instruction labeled *L* is executed next.
5. **Conditional jumps:** **if** *x* **goto** *L* and **ifFalse** *x* **goto** *L*. These execute the instruction labeled *L* next if *x* is true (respectively, false); otherwise, the next instruction in sequence executes.

6. **Relational conditional jumps:** `if x relop y goto L`, which applies a relational operator (`<`, `<=`, `=`, `≠`, `>`, `>=`) to x and y and jumps to L if the relation holds; otherwise execution falls through to the next instruction.
7. **Procedure call / return instructions**, implemented as a sequence:

```
param x1
param x2
...
param xn
call p, n
```

generated for a call $p(x_1, x_2, \dots, x_n)$. Here n records how many actual parameters belong to *this* call, because calls can be nested (some of the first `param` statements might belong to an inner call whose result becomes a parameter of the outer call). A function call that returns a value is written $y = \text{call } p, n$; `return y` returns an (optional) value from a procedure.

8. **Indexed copy instructions:** $x = y[i]$ sets x to the value stored i memory units beyond location y ; $x[i] = y$ sets the memory location i units beyond x to the value of y .
9. **Address and pointer instructions:** $x = \&y$ sets the r-value of x to be the address (l-value) of y ; $x = *y$ sets the r-value of x to the contents of the location pointed to by y ; $*x = y$ sets the r-value of the object pointed to by x to the value of y .

Labels used by jump instructions are symbolic; a label represents the index of a three-address instruction in the instruction sequence. Actual numeric indices can later be substituted for symbolic labels, either in a separate pass or through **backpatching**.

1.4.4 Worked Example — Two Ways of Assigning Labels

Given: the statement `do i = i+1; while (a[i] < v);`, where each array element occupies 8 units of storage.

Translation with symbolic labels:

```
L:  t1 = i + 1
    i  = t1
    t2 = i * 8
    t3 = a[t2]
    if t3 < v goto L
```

The same translation with explicit position numbers (arbitrarily starting at 100):

```
100: t1 = i + 1
101: i  = t1
102: t2 = i * 8
103: t3 = a[t2]
104: if t3 < v goto 100
```

Both versions represent the same program; the last instruction in both cases is a conditional jump back to the *first* instruction, implementing the `do-while` loop. The multiplication `i * 8` computes the byte offset of `a[i]`, appropriate for an array whose elements each take 8 units of space (this offset calculation is developed formally in Part 2 of this note).

1.4.5 Exam Revision Questions

Q1. Define three-address code. What is the maximum number of operators allowed per instruction?

Answer: Three-address code is an intermediate representation in which each instruction contains at most one operator besides the assignment; a typical instruction is $x = y \text{ op } z$, with two operand addresses and one result address (hence “three-address”). Expressions with multiple operators are broken into a sequence of such instructions using temporaries.

Q2. Why does three-address code use compiler-generated temporary names?

Answer: Every intermediate value produced while evaluating a multi-operator expression needs somewhere to live until it is consumed. A fresh temporary is created for each such value so that (a) the unraveled instructions each have only one operator, and (b) in an optimizing compiler, the instructions can later be freely reordered or the temporaries combined into machine registers, since each temporary has a single, well-defined point of definition.

Q3. List and briefly describe the standard forms of three-address instructions.

Answer: (1) binary assignment $x = y \text{ op } z$; (2) unary assignment $x = \text{op } y$; (3) copy $x = y$; (4) unconditional jump `goto L`; (5) Boolean conditional jumps `if x goto L / ifFalse x goto L`; (6) relational conditional jumps `if x relop y goto L`; (7) procedure call/return via `param`, `call p, n`, and `return`; (8) indexed copies $x = y[i]$ and $x[i] = y$; (9) address/pointer operations $x = \&y$, $x = *y$, $*x = y$.

Q4. In `call p, n`, why is the integer n needed even though it can be counted from the `param` statements?

Answer: Because procedure calls can be nested, some of the `param` statements appearing before `call p, n` may actually belong to a different, inner call whose return value is being used as one of `p`’s own parameters. Recording `n` explicitly with the call removes any ambiguity about how many of the preceding `param` instructions belong to this particular call.

Q5. What is the difference between using symbolic labels and position numbers for three-address instructions?

Answer: A symbolic label (e.g., `L`) is a name attached to an instruction that jump instructions refer to; it must eventually be resolved to an actual instruction address. A position number is the literal index of the instruction in the generated sequence. Compilers typically generate code with symbolic labels first (for readability and because targets may not be known yet) and later substitute actual position numbers, either in a separate pass or via backpatching during code generation.

1.5 Representing Three-Address Code: Quadruples, Triples, and Indirect Triples

The list of instruction forms above describes *what* three-address code contains, but not how it is *stored* inside the compiler’s data structures. Three such data-structure representations are commonly used: **quadruples**, **triples**, and **indirect triples**.

1.5.1 Quadruples

A **quadruple** (or “quad”) is a record with **four fields**: *op*, *arg₁*, *arg₂*, and *result*.

- The *op* field holds an internal code for the operator.
- The instruction $x = y + z$ is stored by placing `+` in *op*, `y` in *arg₁*, `z` in *arg₂*, and `x` in *result*.
- **Exceptions:**
 1. Unary operators (e.g., $x = \text{minus } y$, or a copy $x = y$) do not use *arg₂*.

2. Operators like **param** use neither arg_2 nor *result*.
3. Conditional and unconditional jumps put the target label in the *result* field.

Worked Example — Quadruples for $a = b * -c + b * -c$. A special operator name **minus** is used for unary minus, to distinguish it from the binary minus. The naive translation (before any optimization removes the duplicate computation) is:

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a  = t5
```

Index	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

Why copy instructions like the final $a = t5$ appear: a simple expression translator gives every subexpression its own new temporary and only discovers where the value of the *whole* expression should go once the assignment operator itself is processed. Only a later optimization pass (often using a DAG) can discover that **t5** can simply be renamed **a**, eliminating the copy.

1.5.2 Triples

A **triple** has only **three fields**: *op*, *arg₁*, and *arg₂* — there is no separate *result* field. Instead, a triple representation refers to the *result* of an operation by the **position** (index) of the triple that computed it, rather than by an explicit temporary name. This positional reference is exactly the *value number* idea from Section 1.3.3.

Worked Example — Triples for the same expression $a = b * -c + b * -c$:

Index	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Here (0) means “the result computed by the triple at index 0,” and so on. Note how the final copy statement **a = t5** is encoded by placing **a** in *arg₁* and (4) (a pointer to the addition triple) in *arg₂* of triple 5.

A ternary operation, such as an indexed assignment $x[i] = y$, needs **two** triple entries, since a triple only has two argument fields: one triple computes/holds x and i , and the next uses that triple’s result together with y . Similarly, $x = y[i]$ can be implemented as if it were the two instructions $t = y[i]$ and $x = t$, where t is an implicit compiler temporary that never actually appears as a named field — it is simply the position of the first triple.

Comparison: Quadruples versus Triples

Aspect	Quadruples	Triples
Fields	4: op, arg1, arg2, result	3: op, arg1, arg2
How a result is referenced	By an explicit temporary name	By the position (index) of the triple that produced it
Effect of moving an instruction (e.g., during optimization)	No change needed elsewhere — the temporary name travels with the instruction	Every triple that refers to the moved triple’s <i>position</i> must be updated — a real drawback
Storage	Slightly larger per record	Slightly smaller per record

1.5.3 Indirect Triples

Indirect triples consist of a list of *pointers to* triples, rather than a listing of the triples themselves. An auxiliary array, conventionally called **instruction**, lists — in execution order — pointers into the triple table.

Worked Example — Indirect triples for the same expression, assuming the **instruction** list happens to start at index 35:

instruction	->	Index	op	arg1	arg2
35: (0)		0	minus	c	
36: (1)		1	*	b	(0)
37: (2)		2	minus	c	
38: (3)		3	*	b	(2)
39: (4)		4	+	(1)	(3)
40: (5)		5	=	a	(4)

Why indirect triples solve the “moving instructions” problem: With plain triples, reordering an instruction (say, during optimization) means every other triple that references its *position* must be patched. With indirect triples, an optimizer can simply re-order the pointers in the **instruction** list — the triples themselves, and the position-based references *inside* them, never need to change. (When implemented in an object-oriented language such as Java, an array of instruction objects is naturally analogous to indirect triples, since array elements there are references to objects.)

1.5.4 Additional Worked Simulations

The following three simulations (from in-class practice slides) reinforce all three representations side by side, including one example with a procedure call.

Simulation A. Source three-address code:

```

t1 = uminus c
t2 = b * t1
t3 = uminus c
t4 = b * t3
t5 = t2 + t4
a  = t5

```

Quadruples:

Idx	op	arg1	arg2	result
0	uminus	c		t1
1	*	b	t1	t2
2	uminus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

Triples:

Idx	op	arg1	arg2
0	uminus	c	
1	*	b	(0)
2	uminus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Indirect Triples (instruction list starting at 35): 35:(0) 36:(1) 37:(2) 38:(3) 39:(4) 40:(5), with the same triple table as above.

Simulation B (with a procedure call). Source three-address code:

```

t0 = x * y
t1 = t0 + w
a[x] = 5
param w
param k
call foo, 2

```

Quadruples:

Idx	op	arg1	arg2	result
0	*	x	y	t0

Idx	op	arg1	arg2	result
1	+	t0	w	t1
2	=	5	x	a
3	param	w		
4	param	k		
5	call	foo	2	

Triples:

Idx	op	arg1	arg2
0	*	x	y
1	+	(0)	w
2	=	5	x
3	param	w	
4	param	k	
5	call	foo	2

Observe how the indexed assignment $a[x] = 5$ occupies one triple with $op = "="$, $arg1 = 5$, $arg2 = x$ — this is a compact way to encode “store 5 into $a[x]$ ” — and how `param`/`call` naturally fit the three-field triple layout since they need at most two arguments.

Indirect Triples (instruction list starting at 25): 25:(0) 26:(1) 27:(2) 28:(3) 29:(4) 30:(5).

Simulation C. Source three-address code:

```
t1 = b * c
t2 = a + t1
t3 = b * c
t4 = d / t3
t5 = t2 - t4
```

Quadruples:

Idx	op	arg1	arg2	result
0	*	b	c	t1
1	+	a	t1	t2
2	*	b	c	t3
3	/	d	t3	t4
4	-	t2	t4	t5

Triples:

Idx	op	arg1	arg2
0	*	b	c
1	+	a	(0)
2	*	b	c
3	/	d	(2)
4	-	(1)	(3)

Indirect Triples (instruction list starting at 35): 35:(0) 36:(1) 37:(2) 38:(3) 39:(4).

Notice that in Simulation C, the sub-expression `b * c` is computed **twice** (triples 0 and 2), because no DAG-based common-subexpression elimination was applied before generating this code — unlike Section 1.3, where the DAG would have merged these into a single node. This contrast is a good reminder that three-address code by itself does not automatically remove redundant computation; that optimization has to be performed separately (typically using a DAG or a value-numbering table, as in Section 1.3.3).

1.5.5 Exam Revision Questions

Q1. What are the three standard ways of representing three-address code inside a compiler?

Answer: Quadruples (four fields: `op`, `arg1`, `arg2`, `result`), triples (three fields: `op`, `arg1`, `arg2`, with results referenced by position), and indirect triples (a list of pointers into a triple table).

Q2. Compare quadruples and triples in terms of what happens when an optimizer reorders instructions.

Answer: With quadruples, a result is referred to by an explicit temporary name, so moving an instruction requires no changes to any other instruction — the name travels with it. With triples, a result is referred to by the *position* of the triple that computed it, so moving a triple requires updating every other triple that pointed to its old position — a genuine drawback of triples for optimizing compilers.

Q3. How do indirect triples solve the reordering problem of plain triples?

Answer: Indirect triples add a level of indirection: a separate `instruction` list holds pointers, in execution order, to entries of a triple table that never moves. To reorder code, the optimizer only needs to permute the pointers in the `instruction` list; the triples themselves — and the position-based references inside them — remain untouched.

Q4. How is a unary operation, such as `x = minus y`, represented as a quadruple? As a triple?

Answer: As a quadruple: `op = minus`, `arg1 = y`, `arg2` is unused (blank), `result = x`. As a triple: `op = minus`, `arg1 = y`, `arg2` is unused (blank); the value is later referenced by other triples using this triple's position.

Q5. How is the ternary array-store instruction `x[i] = y` represented using triples, given that a triple only has two argument fields?

Answer: It requires two triple entries: the first triple records the base `x` and the index `i` (e.g., an indexed-address triple); the second triple uses the position of the first triple together with `y` to perform the actual store. In general, any operation needing more than two operands must be split across multiple linked triples.

Q6. In the quadruple table for $a = b * -c + b * -c$, why does the last row show $= t5 -- a$ (a copy) instead of directly computing the addition into a ?

Answer: A straightforward, unoptimized expression-translation scheme always creates a brand-new temporary for the value of every subexpression, and only learns the final destination (a) once the enclosing assignment statement itself is translated. It therefore first computes the sum into a temporary $t5$ and then copies $t5$ into a . An optimization pass — often based on the DAG for the expression — can later discover that $t5$ and a can be merged, eliminating the redundant copy instruction.

1.6 Static Single-Assignment (SSA) Form

Static single-assignment (SSA) form is an intermediate representation that facilitates certain code optimizations. It differs from ordinary three-address code in one key respect: **every assignment in SSA is to a variable with a distinct, unique name** — hence the name “static single assignment.” Whenever the source program assigns to a variable more than once, SSA form creates a fresh subscripted version of that variable for each definition.

Worked Example. The following three-address code assigns to p and q more than once each:

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

Its SSA form gives each definition of p and q a distinct subscript:

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

Handling control-flow merges: the ϕ -function. The same variable can be defined along two different control-flow paths. For example:

```
if (flag) x = -1; else x = 1;
y = x * a;
```

There are two possible definitions of x reaching the use in $y = x * a$ — one from the true branch, one from the false branch — so a single SSA name cannot be assigned to x in the ordinary way. SSA form resolves this using a special notational device called the ϕ -**function**, which merges the two incoming definitions:

```
if (flag) x1 = -1; else x2 = 1;
x3 = phi(x1, x2);
```

Here $\phi(x_1, x_2)$ evaluates to x_1 if control reached this point through the true branch of the conditional, and to x_2 if control reached it through the false branch. In other words, the ϕ -function returns the value of whichever argument corresponds to the control-flow path actually taken to reach the assignment statement that contains it.

1.6.1 Exam Revision Questions

Q1. What distinguishes static single-assignment (SSA) form from ordinary three-address code?

Answer: In SSA form, every variable is assigned a value at most once (statically, in the program text). If the original program assigns to the same variable more than once, SSA form introduces a distinct subscripted name for each definition, so that each name has exactly one assignment.

Q2. Why does SSA form need a ϕ -function, and what does it compute?

Answer: When a variable is defined along two different control-flow paths that later merge (e.g., the two branches of an if-else), a single-assignment representation cannot give the merged use point one unique reaching definition using ordinary assignment alone. The ϕ -function is a notational device placed at the merge point that combines the two (or more) incoming SSA names; $\phi(x_1, x_2)$ evaluates to x_1 if control arrived via the path that defined x_1 , and to x_2 if control arrived via the path that defined x_2 .

Q3. Convert the following three-address code into SSA form: $p = a+b$, $q = p-c$, $p = q*d$, $p = e-p$, $q = p+q$.

Answer:

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

Each successive definition of p ($p1$, $p2$, $p3$) and q ($q1$, $q2$) receives a new subscript, and each use refers to the most recent subscript reaching that point.

1.7 Practice Problems

1.7.1 Problem 1 (Spring 2025, Q2)

Convert the following statements into three-address code and show the quadruple representation of it:

```
a = c - b * t * ( - b );
getNeuronBias(( c + d ) * ( - c ), b * ( x / t ));
```

Solution. Unary minus is translated with the operator `uminus`; a function call is translated as a sequence of `param` instructions followed by a `call` instruction. Multiplication is left-associative, so `b * t * (-b)` parses as `(b*t) * (-b)`.

Translating `a = c - b*t*(-b);`:

Step	Subexpression	Action	Generated Code
1	$-b$	unary minus of b into t_1	<code>t1 = uminus b</code>
2	$b * t$	multiply into t_2	<code>t2 = b * t</code>
3	$(b*t) * (-b)$	multiply t_2 and t_1 into t_3	<code>t3 = t2 * t1</code>
4	$c - (b*t*(-b))$	subtract t_3 from c into t_4	<code>t4 = c - t3</code>
5	$a = \dots$	assign t_4 to a	<code>a = t4</code>

Translating `getNeuronBias((c+d)*(-c), b*(x/t));:`

Step	Subexpression	Action	Generated Code
6	$c + d$	add into t_5	<code>t5 = c + d</code>
7	$-c$	unary minus of c into t_6	<code>t6 = uminus c</code>
8	$(c+d) * (-c)$ (1st argument)	multiply t_5, t_6 into t_7	<code>t7 = t5 * t6</code>
9	x / t	divide into t_8	<code>t8 = x / t</code>
10	$b * (x/t)$ (2nd argument)	multiply b, t_8 into t_9	<code>t9 = b * t8</code>
11	pass 1st argument	—	<code>param t7</code>
12	pass 2nd argument	—	<code>param t9</code>
13	invoke the function with 2 arguments	—	<code>call getNeuronBias, 2</code>

Final three-address code (13 instructions):

```

t1 = uminus b
t2 = b * t
t3 = t2 * t1
t4 = c - t3
a = t4
t5 = c + d
t6 = uminus c
t7 = t5 * t6
t8 = x / t
t9 = b * t8
param t7
param t9
call getNeuronBias, 2

```

Quadruple representation:

Idx	op	arg1	arg2	result
0	uminus	b		t1
1	*	b	t	t2
2	*	t2	t1	t3
3	-	c	t3	t4
4	=	t4		a
5	+	c	d	t5
6	uminus	c		t6
7	*	t5	t6	t7
8	/	x	t	t8
9	*	b	t8	t9
10	param	t7		
11	param	t9		
12	call	getNeuronBias	2	

2 Array Access Logic and SDT for Array References

2.1 Learning Introduction

What this topic is. This part explains how a compiler translates references to array elements — both reads ($x = a[i]$) and writes ($a[i] = x$) — into three-address code, and, for multi-dimensional arrays, how the byte address of an element such as $a[i][j][k]$ is computed from its indices.

Why it is needed. Unlike a simple scalar variable, an array element does not have a fixed compile-time address; its address depends on run-time index values. The compiler must therefore generate explicit arithmetic — multiplications and additions — that computes the element’s address at run time, before it can be read from or written to. Getting this address arithmetic right, especially for multi-dimensional arrays, is one of the more intricate parts of intermediate-code generation.

Where it fits in the compilation process. Array-reference translation is part of “Translation of Expressions” (the same phase that turns $a + b$ into three-address code); it reuses the general expression-translation machinery introduced in Part 1 and extends it with a new nonterminal, L , dedicated to array (l-value) references.

2.2 Prerequisite: Translating Simple Expressions

Before array references can be handled, we need the basic rules for translating ordinary (non-array) expressions into three-address code, since array-index expressions are themselves expressions.

For an assignment statement $S \rightarrow \mathbf{id} = E$; and an expression E , we use two attributes: $E.addr$, the address that will hold the value of E , and $E.code$, the three-address code that computes it.

Production	Semantic Rules
$S \rightarrow \mathbf{id} = E$;	$S.code = E.code \parallel$ $gen(top.get(\mathbf{id.lexeme}) ' = ' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new Temp}(); \quad E.code = E_1.code \parallel$ $E_2.code \parallel$ $gen(E.addr ' = ' E_1.addr ' + ' E_2.addr)$

Production	Semantic Rules
$E \rightarrow -E_1$	$E.addr = \mathbf{new\ Temp}(); \quad E.code = E_1.code \parallel \mathbf{gen}(E.addr \mathbf{'=' 'minus' } E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr; \quad E.code = E_1.code$
$E \rightarrow \mathbf{id}$	$E.addr = \mathbf{top.get(id.lexeme)}; \quad E.code = \mathbf{''}$

Here **top** is the current symbol table and **top.get** retrieves the symbol-table entry for an identifier's lexeme; the notation **gen(...)** builds (and, in the incremental scheme, immediately emits) one three-address instruction, evaluating any expressions passed to it and treating quoted strings literally. The $\parallel$ operator concatenates code fragments.

Worked Example. For $a = b + -c$; these rules generate:

```
t1 = minus c
t2 = b + t1
a  = t2
```

In practice, since **code** attributes can be long strings, they are usually built **incrementally** — **gen** both constructs an instruction *and* appends it directly to the output sequence, so the **code** attribute itself is no longer needed as a piece of data being passed around. This incremental style is exactly what the array-reference SDT below uses.

2.2.1 Exam Revision Questions

Q1. What do the attributes $E.addr$ and $E.code$ represent in expression translation?

Answer: $E.addr$ is the address (a name, constant, or temporary) that will hold the value of expression E once its code has executed. $E.code$ is the sequence of three-address instructions that must be executed to compute that value.

Q2. What is meant by “incremental” translation of expressions?

Answer: Instead of building up long **code** strings that are concatenated together and returned as an attribute, the **gen** function directly appends each newly created three-address instruction to a single, growing output sequence as soon as it is generated. This avoids the overhead of manipulating large string-valued attributes.

2.3 Addressing Array Elements

2.3.1 One-Dimensional Arrays

Array elements are stored in a block of **consecutive** memory locations so that any element can be located quickly by arithmetic on its index, rather than by search. Following C and Java convention, the elements of an array with n elements are numbered $0, 1, \dots, n-1$.

Deriving the formula. If each array element occupies w storage units (its *width*), then element 0 occupies units $[base, base + w)$, element 1 occupies $[base + w, base + 2w)$, and in general element i begins exactly i element-widths past the start of the array. Hence the address of $A[i]$ is:

$$\boxed{address(A[i]) = base + i \times w} \quad (1)$$

where $base$ is the relative address of $A[0]$ — the first storage unit allocated to the array.

2.3.2 Generalizing to Multiple Dimensions

Two dimensions. In C/Java, $A[i_1][i_2]$ denotes element i_2 within row i_1 . Let w_1 be the width of an entire row and w_2 the width of a single element within a row. Reasoning exactly as before — move i_1 whole rows past the base, then i_2 elements further within the target row:

$$\text{address}(A[i_1][i_2]) = \text{base} + i_1 \times w_1 + i_2 \times w_2 \quad (2)$$

k dimensions. The same reasoning, applied one dimension at a time, generalizes to:

$$\text{address}(A[i_1][i_2] \cdots [i_k]) = \text{base} + i_1 \times w_1 + i_2 \times w_2 + \cdots + i_k \times w_k \quad (3)$$

where w_j ($1 \leq j \leq k$) is the width of everything varying only in dimensions $j, j+1, \dots, k$ — i.e., the size of a “slab” fixed in the first $j-1$ indices.

Alternative form, using element counts. Instead of widths w_j , we can express the same address using the *number of elements* n_j along each dimension j , and the width w of a single (scalar) array element. In two dimensions ($k=2$, so $w=w_2$):

$$\text{address}(A[i_1][i_2]) = \text{base} + (i_1 \times n_2 + i_2) \times w \quad (4)$$

In k dimensions, this generalizes (by nesting the same idea) to:

$$\text{address}(A[i_1] \cdots [i_k]) = \text{base} + ((\cdots (i_1 \times n_2 + i_2) \times n_3 + i_3) \cdots) \times n_k + i_k) \times w \quad (5)$$

Both forms (widths w_j , or counts n_j with a single scalar width w) compute exactly the same address; the translation scheme used later in this chapter (Fig. 6.22 / Section “SDT for Array References”) uses the **width-based** form, Formula (3).

2.3.3 Row-Major versus Column-Major Layout

The address formulas above assume **row-major** layout — arrays are stored row by row, which is the convention in C and Java. The alternative, **column-major** layout (used in the Fortran family of languages), stores arrays column by column.

2×3 array A — Row-Major	2×3 array A — Column-Major
$A[1, 1]$	$A[1, 1]$
$A[1, 2]$	$A[2, 1]$
$A[1, 3]$	$A[1, 2]$
$A[2, 1]$	$A[2, 2]$
$A[2, 2]$	$A[1, 3]$
$A[2, 3]$	$A[2, 3]$

In row-major form, as we scan down the storage block, the **rightmost** subscript varies fastest (like the units digit on an odometer). In column-major form, the **leftmost** subscript varies fastest. Both layouts generalize naturally to more than two dimensions by extending which subscript “varies fastest.”

2.3.4 Arrays Whose Index Does Not Start at 0

More generally, an array's elements need not be numbered starting at 0. If a one-dimensional array's elements are numbered $low, low + 1, \dots, high$, and $base$ is the relative address of $A[low]$, Formula (1) is replaced by:

$$address(A[i]) = base + (i - low) \times w \quad (6)$$

Both Formula (1) and Formula (6) can be rewritten in the common form

$$address(A[i]) = i \times w + c, \quad \text{where } c = base - low \times w \quad (7)$$

Since low , $base$, and w are all known at **compile time** (for a statically-sized array), the constant c can be **precomputed once** and stored in the array's symbol-table entry. Thereafter, the address of any reference $A[i]$ is obtained with a *single* multiplication and addition: $i \times w + c$. (Note $c = base$ exactly when $low = 0$, which is the common C/Java case.) This same precomputation trick generalizes to multidimensional arrays.

Important exception — dynamically sized arrays. If the array's bounds (low , $high$, or their multidimensional generalizations) are **not known at compile time**, the constant c cannot be precomputed; Formula (6)/(7)-style expressions must then be evaluated in full at run time, when the bounds become known.

Symbolic Type Widths

The intermediate code should ideally be independent of the target machine, so that the optimizer does not need to change when the code generator is retargeted. The translation schemes here assume, for example, that an integer occupies 4 bytes, which is a machine-dependent assumption. Some intermediate codes (e.g., P-code for Pascal) instead leave the *size* of array elements to be filled in later by the code generator, keeping the intermediate code itself independent of a particular machine's word size. This could equally be done here by replacing the literal width (e.g., 4) with a symbolic constant that the back end resolves.

2.3.5 Exam Revision Questions

Q1. Derive the formula for the address of the i^{th} element of a one-dimensional, zero-based array A with element width w .

Answer: Elements are stored consecutively starting at the array's base address. Element 0 occupies the first w units starting at $base$; element 1 starts w units later; in general, element i starts exactly i widths past the base. Hence $address(A[i]) = base + i \times w$.

Q2. Derive the address formula for a two-dimensional array reference $A[i_1][i_2]$ in row-major order.

Answer: In row-major order, all of row $i_1 - 1$'s predecessor rows must be skipped first: that is i_1 complete rows, each of width w_1 , giving offset $i_1 \times w_1$. Within the target row, element i_2 is then reached by moving i_2 element-widths (w_2 each) further. Combining, $address(A[i_1][i_2]) = base + i_1 \times w_1 + i_2 \times w_2$.

Q3. Give the general formula for the address of $A[i_1][i_2] \dots [i_k]$ in a k -dimensional array, in terms of widths $w_1, \dots, w_k$.

Answer: $address = base + i_1 w_1 + i_2 w_2 + \dots + i_k w_k$, where w_j is the width of a "slab" that varies only in dimensions j through k (equivalently, the amount of storage spanned by one unit increase of index i_j while the higher-order indices are held fixed).

Q4. Distinguish row-major and column-major array storage.

Answer: In row-major storage (used by C and Java), array elements are laid out row by row, so the rightmost index varies fastest as we scan through memory. In column-major storage (used by the Fortran family), elements are laid out column by column, so the leftmost index varies fastest.

Q5. How can address computation for a fixed-size array be sped up at compile time, and when does this optimization fail?

Answer: The formula $base + (i - low) \times w$ can be algebraically rewritten as $i \times w + c$, where $c = base - low \times w$ is a constant that depends only on compile-time-known quantities ($base$, low , w). This constant can be computed once and stored in the symbol table, so each array access needs only one multiplication and one addition at run time. This optimization fails when the array's bounds are not known until run time (dynamically sized arrays), because c then cannot be precomputed — the full formula must be evaluated when the program executes and the bounds become available.

Q6. Why might a compiler use a symbolic constant instead of a literal number for an array element's width?

Answer: Using a literal (e.g., 4 bytes for an integer) hard-codes a machine-dependent assumption into the intermediate code, which then must change if the code generator targets a different machine. Using a symbolic constant instead keeps the intermediate representation machine-independent, deferring the actual numeric width to the back end / code generator.

2.4 Grammar and SDT for Array References**2.4.1 The Grammar**

Let nonterminal L generate an array name followed by one or more index expressions:

$$L \rightarrow L[E] \mid \text{id}[E]$$

As in C and Java, we assume the lowest-numbered array element is 0, and we compute addresses using the **width-based** formula (3) from the previous section, rather than the element-count form.

2.4.2 The Semantic Actions

The table below (the complete translation scheme for array references) combines the ordinary expression rules with new rules for L :

Production	Semantic Actions
$S \rightarrow \text{id} = E;$	$gen(top.get(\text{id.lexeme}) \text{ ' } = \text{ ' } E.addr)$
$S \rightarrow L = E;$	$gen(L.array.base \text{ ' } [\text{ ' } L.addr \text{ ' }] \text{ ' } = \text{ ' } E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}(); \quad gen(E.addr \text{ ' } = \text{ ' } E_1.addr \text{ ' } + \text{ ' } E_2.addr)$
$E \rightarrow \text{id}$	$E.addr = top.get(\text{id.lexeme})$
$E \rightarrow L$	$E.addr = \text{new Temp}(); \quad gen(E.addr \text{ ' } = \text{ ' } L.array.base \text{ ' } [\text{ ' } L.addr \text{ ' }] \text{ ' })$
$L \rightarrow \text{id}[E]$	$L.array = top.get(\text{id.lexeme}); \quad L.type = L.array.type.elem; \quad L.addr = \text{new Temp}(); \quad gen(L.addr \text{ ' } = \text{ ' } E.addr \text{ ' } * \text{ ' } L.type.width)$

Production	Semantic Actions
$L \rightarrow L_1 [E]$	$L.array = L_1.array;$ $L.type =$ $L_1.type.elem;$ $t = \mathbf{new} Temp();$ $L.addr =$ $\mathbf{new} Temp();$ $gen(t ' ='$ $E.addr ' *' L.type.width);$ $gen(L.addr ' ='$ $L_1.addr ' +' t)$

2.4.3 Meaning of L 's Three Synthesized Attributes

Nonterminal L carries three synthesized attributes that together let the compiler build up an array element's address one index at a time:

1. $L.addr$ — a temporary that accumulates the running **offset** for the array reference, i.e., the partial sum of the terms $i_j \times w_j$ from Formula (3), computed so far.
2. $L.array$ — a pointer to the symbol-table entry for the array's name, so that $L.array.base$ (the array's base address, the address of its element 0) can be found once *all* the index expressions have been analyzed.
3. $L.type$ — the type of the sub-array denoted by L at this point in the derivation. For any type t , we assume $t.width$ gives its storage width, and for an array type t , $t.elem$ gives the type of its elements. Types (not raw widths) are carried as attributes because they are needed anyway for type checking.

2.4.4 Explanation, Production by Production

- $S \rightarrow \mathbf{id} = E;$ — an assignment to a plain (non-array) variable; unchanged from the basic expression scheme.
- $S \rightarrow L = E;$ — an assignment *to* an array element. Once L 's index expressions have all been processed, $L.array$ names the array and $L.array.base$ is its base address; $L.addr$ holds the computed offset. The generated instruction is an **indexed copy** that stores $E.addr$'s r-value into location $L.array.base[L.addr]$.
- $E \rightarrow E_1 + E_2$ **and** $E \rightarrow \mathbf{id}$ — unchanged from the ordinary expression rules.
- $E \rightarrow L$ — this is a *read* of an array element used inside a larger expression. It copies the r-value found at $L.array.base[L.addr]$ into a brand-new temporary, which becomes $E.addr$.
- $L \rightarrow \mathbf{id} [E]$ (**the base case**) — this is the *first* (leftmost) index bracket applied to an array name. $L.array$ is set to the array's own symbol-table entry, $L.type$ becomes the type of one "slice" obtained by removing the first index (i.e., **array.type.elem**), and the initial offset is computed as $E.addr \times L.type.width$ — exactly the first term $i_1 \times w_1$ of Formula (3).
- $L \rightarrow L_1 [E]$ (**the recursive case**) — for a *subsequent* index bracket (second, third, ...). $L.array$ is inherited unchanged from L_1 . $L.type$ narrows further by one more **elem**. The new index's contribution, $E.addr \times L.type.width$, is computed into a fresh temporary t , and then **added** to the offset accumulated so far ($L_1.addr$) to produce the new running offset $L.addr$. This is exactly how Formula (3)'s sum $i_1 w_1 + i_2 w_2 + \dots$ is built up term by term, one bracket at a time.

2.4.5 Exam Revision Questions

Q1. Give the grammar used for array references in the SDT, and explain what each production represents.

Answer: $L \rightarrow L [E] \mid \mathbf{id} [E]$. The base production $\mathbf{id} [E]$ represents the first index applied directly to an array name; the recursive production $L_1 [E]$ represents each additional index bracket applied to the result of a previous (sub)array reference, allowing any number of dimensions to be handled uniformly.

Q2. List and explain the three synthesized attributes of nonterminal L in the array-reference SDT.

Answer: $L.addr$ is a temporary holding the array reference's accumulated byte offset so far. $L.array$ points to the symbol-table entry of the array being indexed, giving access to its base address. $L.type$ is the type of the sub-array represented by L at the current point (used to obtain the element width $L.type.width$ for the next index, and needed anyway for type checking).

Q3. Why does the production $L \rightarrow L_1[E]$ generate two three-address instructions instead of one?

Answer: It must (a) compute the *contribution* of this new index to the offset, $t = E.addr \times L.type.width$, and (b) *add* that contribution to the offset already accumulated from the earlier indices, $L.addr = L_1.addr + t$. Both steps are needed because Formula (3)'s address is a running sum built up index by index, one bracket at a time, rather than a single multiplication.

Q4. What is the difference between the semantic actions for $E \rightarrow L$ and $S \rightarrow L = E$;?

Answer: $E \rightarrow L$ *reads* the array element: it copies the r-value at $L.array.base[L.addr]$ into a new temporary, which becomes the value of the expression. $S \rightarrow L = E$; *writes* to the array element: it generates an indexed-copy instruction that stores $E.addr$'s value into $L.array.base[L.addr]$. One is a load, the other is a store, and both rely on the same $L.array.base/L.addr$ pair produced by the L productions.

Q5. Why does the production $L \rightarrow id[E]$ use $L.array.type.elem$ for $L.type$ instead of the array's own type?

Answer: Once one index has been applied, the result denotes a *sub-array* (or, for a 1-D array, a scalar element) — one dimension “less” than the original array. `type.elem` gives precisely the element type of an array type, so $L.type$ correctly reflects what remains to be indexed by any further brackets, and correctly gives the *width to multiply by* for the current index.

2.5 Worked Simulations of Array-Reference Translation

Each simulation below verifies the generated code directly against the address formulas from the previous section.

2.5.1 Simulation 1 — One-Dimensional Array

Given: a is a one-dimensional array of integers with 8 elements, so its type is `array(8, integer)` and its width is $w = 32$ (since an integer occupies 4 bytes: $8 \times 4 = 32$). c, i are integers. Translate the expression $c + a[i]$.

Step-by-step parse and translation:

Step	Subtree	Action	Generated Code
1	$E \rightarrow id$ (for c)	$E_1.addr = c$ (symbol-table entry); no code generated	—
2	$L \rightarrow id[E]$ (for $a[i]$)	$L.array = a$, $L.type = integer$; compute offset = $i \times 4$ into t_1	$t1 = i * 4$
3	$E \rightarrow L$ (wraps $a[i]$ as a value)	copy $a[t_1]$ into a new temporary t_2	$t2 = a[t1]$

Step	Subtree	Action	Generated Code
4	$E \rightarrow E_1 + E_2$ ($c + a[i]$)	add c and t_2 into a new temporary t_3	t3 = c + t2

Final three-address code:

```
t1 = i * 4
t2 = a[t1]
t3 = c + t2
```

Verification against the formula. Formula (1) gives $address(a[i]) = base + i \times w = base + i \times 4$. Instruction **t1 = i*4** computes exactly the offset $i \times w$; instruction **t2 = a[t1]** then performs the indexed load $a.base[t_1]$, which is precisely $address(a[i])$. The result matches the formula, confirming the translation is correct.

2.5.2 Simulation 2 — Two-Dimensional Array (Example 6.12)

Given: a is a 2×3 array of integers, so its type is $array(2, array(3, integer))$, with overall width $w = 24$. The type of $a[i]$ is $array(3, integer)$ of width $w_1 = 12$; the type of $a[i][j]$ is $integer$, of width $w_2 = 4$. Translate $c + a[i][j]$.

Step	Subtree	Action	Generated Code
1	$E \rightarrow \mathbf{id}$ (c)	$E_1.addr = c$	—
2	$L \rightarrow \mathbf{id}[E]$ ($a[i]$, first bracket)	$L_1.array = a$, $L_1.type = array(3, integer)$ ($w_1 = 12$); offset $= i \times 12$ into t_1	t1 = i * 12
3	$L \rightarrow L_1[E]$ ($a[i][j]$, second bracket)	$L.array = a$, $L.type = integer$ ($w_2 = 4$); contribution $= j \times 4$ into t_2 ; add to running offset into t_3	t2 = j * 4 then t3 = t1 + t2
4	$E \rightarrow L$ (wraps $a[i][j]$ as a value)	copy $a[t_3]$ into new temporary t_4	t4 = a[t3]
5	$E \rightarrow E_1 + E_2$ ($c + a[i][j]$)	add c and t_4 into new temporary t_5	t5 = c + t4

Final three-address code:

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = c + t4
```

Verification against the formula. Formula (2) gives $address(a[i][j]) = base + i \times w_1 + j \times w_2 = base + i \times 12 + j \times 4$. Steps 1–3 above compute exactly $t_3 = i \times 12 + j \times 4$, the offset part of the formula; step 4's $a[t_3]$ performs the indexed load at $base[t_3]$, matching the formula exactly.

2.5.3 Simulation 3 — Three-Dimensional Array

Given: a is a $2 \times 3 \times 4$ array of integers, type $array(2, array(3, array(4, integer)))$, overall width $w = 96$. The type of $a[i]$ is $array(3, array(4, integer))$, width $w_1 = 48$; the type of $a[i][j]$ is $array(4, integer)$, width $w_2 = 16$; the type of $a[i][j][k]$ is $integer$, width $w_3 = 4$. Translate the statement $ab = a[i][j][k];$.

Step	Subtree	Action	Generated Code
1	$L \rightarrow id[E] (a[i])$	$L_1.type =$ $array(3, array(4, integer))$ ($w_1 = 48$); offset $= i \times 48$ into t_1	$t1 = i * 48$
2	$L \rightarrow L_1[E]$ $(a[i][j])$	$L_2.type =$ $array(4, integer)$ ($w_2 = 16$); contribution $= j \times 16$ into t_2 ; running offset into $t_3 = t_1 + t_2$	$t2 = j * 16$ then $t3$ $= t1 + t2$
3	$L \rightarrow L_2[E]$ $(a[i][j][k])$	$L_3.type = integer$ ($w_3 = 4$); contribution $= k \times 4$ into t_4 ; running offset into $t_5 = t_3 + t_4$	$t4 = k * 4$ then $t5 =$ $t3 + t4$
4	$E \rightarrow L$	copy $a[t_5]$ into new temporary t_6	$t6 = a[t5]$
5	$S \rightarrow id = E;$	assign t_6 to ab	$ab = t6$

Final three-address code:

```

t1 = i * 48
t2 = j * 16
t3 = t1 + t2
t4 = k * 4
t5 = t3 + t4
t6 = a[t5]
ab = t6

```

Verification against the formula. Formula (3) with $k = 3$ gives $address(a[i][j][k]) = base + i \times 48 + j \times 16 + k \times 4$. Adding up the contributions built incrementally in steps 1–3 gives $t_5 = i \times 48 + j \times 16 + k \times 4$ — exactly the offset term of the formula — confirming the translation is correct. Note how each bracket contributes exactly one multiplication (index $\times$ its slice width) and, after the first bracket, one addition to fold the new contribution into the running total; this is the general pattern for any number of dimensions.

2.5.4 Extra Practice — Sum of Two Array Reads

Translate: $x = a[i] + b[j];$, where a and b are one-dimensional integer arrays with element width 4.

Step	Subtree	Action	Generated Code
1	$L \rightarrow \text{id}[E] (a[i])$	offset = $i \times 4$ into t_1	$t1 = i * 4$
2	$E \rightarrow L$	load $a[t_1]$ into t_2	$t2 = a[t1]$
3	$L \rightarrow \text{id}[E] (b[j])$	offset = $j \times 4$ into t_3	$t3 = j * 4$
4	$E \rightarrow L$	load $b[t_3]$ into t_4	$t4 = b[t3]$
5	$E \rightarrow E_1 + E_2$	add t_2 and t_4 into t_5	$t5 = t2 + t4$
6	$S \rightarrow \text{id} = E;$	assign to x	$x = t5$

Final code:

```
t1 = i * 4
t2 = a[t1]
t3 = j * 4
t4 = b[t3]
t5 = t2 + t4
x = t5
```

This example shows that the two array reads $a[i]$ and $b[j]$ are completely independent sub-derivations of L — each generates its own offset temporary and its own load — and are simply combined by the ordinary addition rule once both have produced a value in $E.addr$.

2.5.5 Exam Revision Questions

Q1. Simulate the translation of $x = a[i]$, for a one-dimensional integer array a (element width 4), showing every step.

Answer: Step 1: apply $L \rightarrow \text{id}[E]$ — compute the offset $t1 = i * 4$. Step 2: apply $E \rightarrow L$ — load the element, $t2 = a[t1]$. Step 3: apply $S \rightarrow \text{id} = E;$ — assign, $x = t2$. Final code: $t1 = i*4, t2 = a[t1], x = t2$.

Q2. Why does translating a two-dimensional array reference $a[i][j]$ require an addition instruction that a one-dimensional reference $a[i]$ does not?

Answer: A one-dimensional reference has only one index, so its offset is a single product $i \times w$ with nothing to add it to. A two-dimensional reference has two index contributions, $i \times w_1$ and $j \times w_2$, that must be summed to obtain the total offset, per Formula (2); this summation is exactly what the extra add instruction (generated by the recursive production $L \rightarrow L_1[E]$) performs.

Q3. For a 3-D array with per-dimension widths w_1, w_2, w_3 , how many multiplication and how many addition instructions does the SDT of this section generate to compute the address offset of $a[i][j][k]$?

Answer: Three multiplications (one per index: $i \times w_1, j \times w_2, k \times w_3$) and two additions (folding the second and third contributions into the running offset). In general, for a k -dimensional array, the scheme generates k multiplications and $k - 1$ additions.

Q4. In Simulation 2, why is $t1 = i*12$ computed using width 12, while $t2 = j*4$ uses width 4?

Answer: t_1 is the offset contribution of the *first* index i , which selects a whole row; since each row is a sub-array of type $\text{array}(3, \text{integer})$, its width is $3 \times 4 = 12$ bytes, matching $L.\text{type.width}$ for the base production $L \rightarrow \text{id}[E]$. t_2 is the contribution of the *second* index j , which selects a single integer element within that row, so its width is just 4 bytes — the element type’s width — matching $L.\text{type.width}$ after one more .elem narrowing in the recursive production $L \rightarrow L_1[E]$.

2.6 Practice Problems

2.6.1 Problem 1 (Fall 2025, Q3)

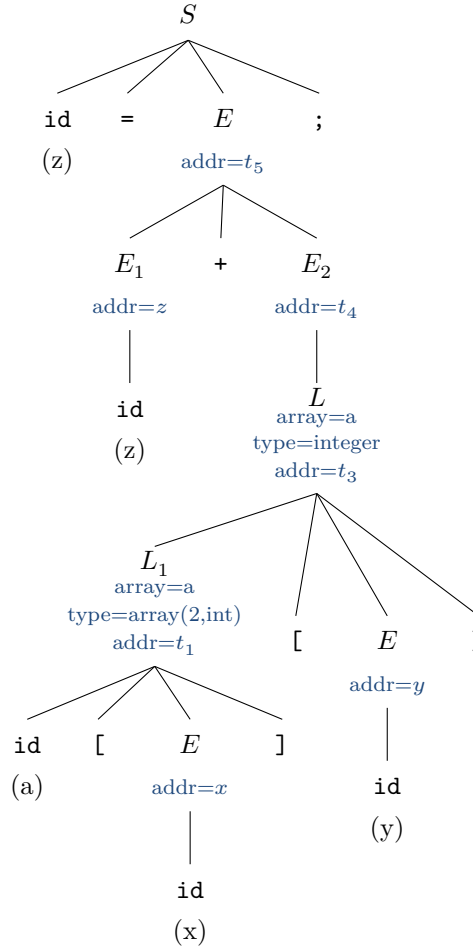
Draw the annotated parse tree, evaluate the tree, and write the three-address code for the string:

`z = z + a[x][y];`

a is a two-dimensional integer array, type $\text{array}(3, \text{array}(2, \text{integer}))$, overall width $w = 24$. The type of $a[x]$ is $\text{array}(2, \text{integer})$, of width $w_1 = 8$. The type of $a[x][y]$ is integer .

Solution.

Annotated Parse Tree:



Step-by-step parse and translation (this is $S \rightarrow \text{id} = E;$ with $\text{id}=z$ and $E \rightarrow E_1 + E_2$, $E_1 = z$, and E_2 is the array read $a[x][y]$):

Step	Subtree	Action	Generated Code
1	$E_1 \rightarrow \mathbf{id} (z, \text{first occurrence})$	$E_1.addr = z$ (symbol-table entry); no code	—
2	$L \rightarrow \mathbf{id}[E] (a[x], \text{first bracket})$	$L_1.array = a,$ $L_1.type =$ $array(2, integer)$ ($w_1 = 8$); offset $= x \times 8$ into t_1	$t1 = x * 8$
3	$L \rightarrow L_1[E] (a[x][y], \text{second bracket})$	$L.type = integer$ ($w_2 = 4$); contribution $= y \times 4$ into t_2 ; add to running offset into t_3	$t2 = y * 4$ then $t3 =$ $t1 + t2$
4	$E_2 \rightarrow L$ (wraps $a[x][y]$ as a value)	copy $a[t_3]$ into new temporary t_4	$t4 = a[t3]$
5	$E \rightarrow E_1 + E_2 (z +$ $a[x][y])$	add z and t_4 into new temporary t_5	$t5 = z + t4$
6	$S \rightarrow \mathbf{id} = E; (z =$ $\dots)$	assign t_5 to z	$z = t5$

Final three-address code:

```

t1 = x * 8
t2 = y * 4
t3 = t1 + t2
t4 = a[t3]
t5 = z + t4
z = t5

```

2.6.2 Problem 2 (Spring 2026, Q6)

Using the grammar and semantic rules given, draw the parse tree and write the generated three-address code for:

```

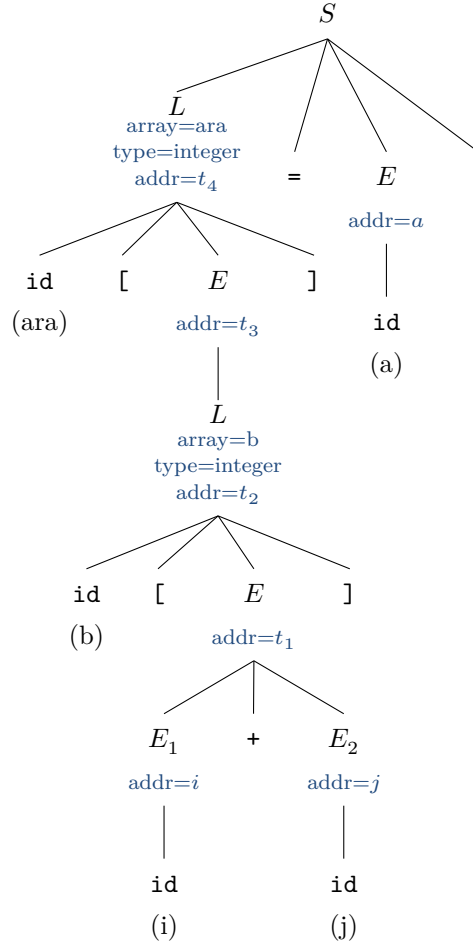
ara[b[i + j]] = a;

```

ara and **b** are integer arrays of length 15, and each integer occupies 4 bytes; **i**, **j**, **a** are integers.

Solution. The index expression itself is an array reference: **b[i+j]** must first be **read** (via $E \rightarrow L$) before its value can be used as the index of the outer array **ara**. This exercises $L \rightarrow \mathbf{id}[E]$ (build an address), $E \rightarrow L$ (load through that address), and $S \rightarrow L = E;$ (store through an address) within a single statement.

Annotated Parse Tree:



Step-by-step parse and translation (this is $S \rightarrow L = E ;$, where the left-hand L is `ara[b[i+j]]` and the right-hand E is `a`):

Step	Subtree	Action	Generated Code
1	$E \rightarrow E_1 + E_2$ (<code>i + j</code> , the inner index expression)	add <code>i</code> , <code>j</code> into new temporary t_1	<code>t1 = i + j</code>
2	$L \rightarrow \text{id}[E]$ (<code>b[i+j]</code>)	$L_{inner}.array = b$, $L_{inner}.type = integer$ ($w = 4$); offset $= t_1 \times 4$ into t_2	<code>t2 = t1 * 4</code>
3	$E \rightarrow L$ (wraps <code>b[i+j]</code> as a value — this is the index used by the <i>outer</i> array)	copy $b[t_2]$ into new temporary t_3	<code>t3 = b[t2]</code>
4	$L \rightarrow \text{id}[E]$ (<code>ara[...]</code> , using t_3 as the index)	$L.array = ara$, $L.type = integer$ ($w = 4$); offset $= t_3 \times 4$ into t_4	<code>t4 = t3 * 4</code>
5	$E \rightarrow \text{id}$ (<code>a</code> , right-hand side)	$E.addr = a$ (symbol-table entry); no code	—

Step	Subtree	Action	Generated Code
6	$S \rightarrow L = E$; (the outer assignment)	store $E.addr$ into $ara[t_4]$	ara[t4] = a

Final three-address code:

```
t1 = i + j
t2 = t1 * 4
t3 = b[t2]
t4 = t3 * 4
ara[t4] = a
```

2.6.3 Problem 3 (Spring 2025, Q3)

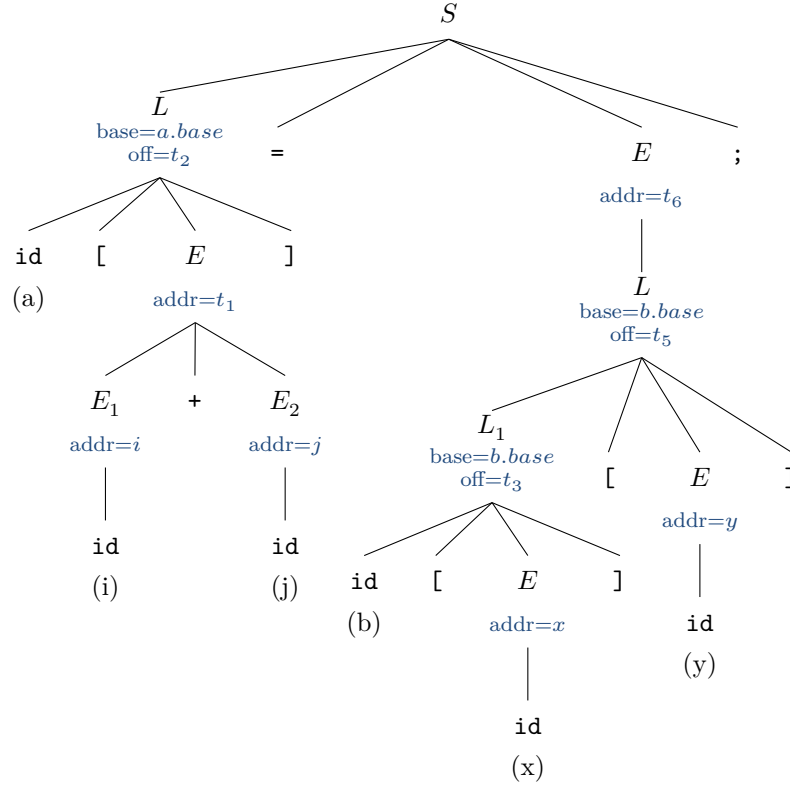
Draw the annotated parse tree and evaluate it for the string (annotating **addr**, **type**, **idx**, **off**, and the value of **tScale** where it is updated):

```
a[i+j] = b[x][y];
```

a is a one-dimensional integer array of size 10, type $array(10, integer)$, width $w = 40$ (so its element width is 4). **b** is a two-dimensional integer array of size 5×7 , type $array(5, array(7, integer))$, width $w = 140$ (so its row width is $140/5 = 28$, and its element width is $28/7 = 4$).

Solution. This problem's SDT uses the attribute names $L.base$ (array base address), $L.idx$ (the current bracket's raw index value), and $L.off$ (the running accumulated offset), together with a local variable $tScale$ used inside $L \rightarrow L_1[E]$ to hold one index's scaled contribution before it is added into $L.off$:

Production	Semantic Actions (as given in the exam)
$S \rightarrow id = E$;	$gen(top.get(id.lexeme) ' = ' E.addr)$
$S \rightarrow L = E$;	$gen(L.base '[' L.off '] ' = ' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new Temp}(); gen(E.addr ' = ' E_1.addr + E_2.addr)$
$E \rightarrow id$	$E.addr = top.get(id.lexeme)$
$E \rightarrow L$	$E.addr = \mathbf{new Temp}(); gen(E.addr ' = ' L.base[L.off])$
$L \rightarrow id[E]$	$L.array = top.get(id.lexeme); L.type = L.array.type.elem; L.base = L.array.base; L.idx = E.addr; L.off = \mathbf{new Temp}(); gen(L.off = L.idx \times L.type.width)$
$L \rightarrow L_1[E]$	$L.array = L_1.array; L.type = L_1.type.elem; L.base = L_1.base; L.idx = E.addr; tScale = \mathbf{new Temp}(); gen(tScale = L.idx \times L.type.width); L.off = \mathbf{new Temp}(); gen(L.off = L_1.off + tScale)$

Annotated Parse Tree:

Step-by-step parse and translation. The statement is $S \rightarrow L = E;$, with the left-hand $L = a[i+j]$ and the right-hand $E = b[x][y]$:

Step	Subtree	Action	Generated Code
1	$E \rightarrow E_1 + E_2$ (i+j, the index of the left-hand array)	add i, j into new temporary t_1	$t1 = i + j$
2	$L \rightarrow \text{id}[E]$ ($a[i+j]$)	$L_a.array = a,$ $L_a.type = integer$ ($w = 4$), $L_a.base = a.base,$ $L_a.idx = t_1$; offset $L_a.off = t_1 \times 4$ into t_2	$t2 = t1 * 4$
3	$L \rightarrow \text{id}[E]$ ($b[x]$, first bracket of the right side)	$L_1.array = b,$ $L_1.type = array(7, integer)$ ($w_1 = 28$), $L_1.base = b.base,$ $L_1.idx = x$; offset $L_1.off = x \times 28$ into t_3	$t3 = x * 28$

Step	Subtree	Action	Generated Code
4	$L \rightarrow L_1[E] \text{ (b[x] [y], second bracket)}$	$L_b.type = integer$ $(w_2 = 4), L_b.idx = y;$ $tScale = y \times 4$ into t_4 ; running offset $L_b.off =$ $L_1.off + tScale$ into t_5	$t4 = y * 4$ then $t5 =$ $t3 + t4$
5	$E \rightarrow L$ (wraps $b[x] [y]$ as a value)	copy $b[t_5]$ into new temporary t_6	$t6 = b[t5]$
6	$S \rightarrow L = E$; (the outer assignment)	store $E.addr = t_6$ into $a[L_a.off] = a[t_2]$	$a[t2] = t6$

Final three-address code:

```

t1 = i + j
t2 = t1 * 4
t3 = x * 28
t4 = y * 4
t5 = t3 + t4
t6 = b[t5]
a[t2] = t6

```

3 Handling Flow-of-Control Statements

3.1 Learning Introduction

What this topic is. This part covers how a compiler translates statements that alter the flow of control — **if**, **if-else**, and **while** — together with the Boolean expressions that drive them, into three-address code containing explicit labels and jump instructions.

Why it is needed. Source-level control constructs are structured and hierarchical (an **if**-statement “contains” its body), but target machines only understand flat sequences of instructions with conditional and unconditional jumps. The compiler must therefore translate nested conditionals and loops into equivalent jump-based code that preserves their meaning exactly, while trying to avoid generating unnecessary jumps.

Where it fits in the compilation process. This is the second major sub-topic of “Translation of Statements” within intermediate-code generation (the first being array/expression translation, covered in Part 2). It builds directly on the three-address instruction set from Part 1 — particularly the conditional-jump forms **if x goto L** and **ifFalse x goto L**.

3.2 Boolean Expressions: Two Roles, and a Grammar

In programming languages, Boolean expressions serve **two distinct purposes**:

1. **Altering the flow of control.** Boolean expressions act as the condition in constructs that change which statement executes next — e.g., in **if (E) S**, execution of **S** is conditional on **E** being true. Here, the *only* thing that matters is *where control goes next* — the Boolean value itself is never stored anywhere.

2. **Computing a logical value.** A Boolean expression can also be evaluated the way an arithmetic expression is, producing a **true/false** value that is stored, e.g., in an assignment `x = a < b;`.

Because these two uses are so different in what code they require, compilers typically distinguish between them — using different nonterminals, inherited attributes, syntax-tree flags, or separate code-generation procedures. **This section is chiefly concerned with the first role** — using Boolean expressions to alter the flow of control — and introduces nonterminal B specifically for that purpose. The second role (computing values) is treated separately.

Grammar for Boolean expressions (following C’s operator conventions: `&&` for AND, `||` for OR, `!` for NOT):

$$B \rightarrow B \parallel B \mid B \&\& B \mid !B \mid (B) \mid E \text{ rel } E \mid \text{true} \mid \text{false}$$

Here `rel.op` denotes which of the six comparison operators (`<`, `<=`, `=`, `≠`, `>`, `>=`) is being used. By convention, `||` and `&&` are left-associative, and precedence increases in the order `||`, then `&&`, then `!` (i.e., `!` binds most tightly).

If a language definition permits (or requires) that only part of a Boolean expression be evaluated, the compiler can **short-circuit** evaluation: in $B_1 \parallel B_2$, if B_1 is found to be true, B_2 need not be evaluated at all (the whole expression is already true); similarly, if B_1 in $B_1 \&\& B_2$ is false, B_2 need not be evaluated. (If B_2 has side effects, e.g. it calls a function that changes a global variable, skipping it can change the program’s observable behavior — this is exactly why some languages *require*, and others merely *permit*, short-circuiting.)

3.3 Short-Circuit (Jumping) Code

In **short-circuit** or **jumping** code, the Boolean operators `&&`, `||`, and `!` themselves generate **no instructions at all**. Instead, the *truth value* of a Boolean expression is represented implicitly, by **which label control reaches** as it falls through the generated instruction sequence — not by storing **true/false** in a variable.

Worked Example. The statement

```
if ( x < 100 || x > 200 && x != y ) x = 0;
```

might be translated into:

```
if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2: x = 0
L1:
```

Reading the code: control reaches label L2 — and hence executes `x = 0` — precisely when the whole Boolean expression is **true**; if the expression is **false**, control skips directly to L1, bypassing the assignment. Observe that **no Boolean value is ever computed or stored** — the three relational tests simply jump to one of two places, L2 (true) or L1 (false), directly implementing short-circuit evaluation of the `||` and (implicitly, through precedence) the `&&`.

3.3.1 Exam Revision Questions

Q1. Explain the two roles that Boolean expressions play in a programming language, with an example of each.

Answer: (1) Altering flow of control — e.g. the condition E in `if (E) S`, where only the *destination* of control matters, not a stored value. (2) Computing a logical value — e.g. `x = a < b;`, where the Boolean result is stored into a variable much like an arithmetic result. Compilers often use separate translation strategies for these two uses because the flow-of-control use needs only jumps, while the value-computing use needs an actual stored true/false result.

Q2. What is short-circuit (jumping) code, and what do the Boolean operators translate into?

Answer: Short-circuit code is three-address code in which a Boolean expression's truth value is represented purely by the *position* (label) control reaches, rather than by a stored value. The Boolean operators `&&`, `||`, and `!` generate no code of their own; they only affect which labels the underlying relational tests jump to. This automatically implements short-circuit evaluation, since a sub-expression that is not needed to determine the final result is simply never reached.

Q3. In the jumping-code translation of `if (x<100 || x>200 && x!=y) x=0;`, under what condition does control reach L2, and under what condition does it reach L1 directly?

Answer: Control reaches L2 (and executes `x=0`) exactly when the Boolean expression `x<100 || (x>200 && x!=y)` evaluates to true — i.e., either `x<100` holds, or both `x>200` and `x!=y` hold. Control jumps directly to L1, skipping the assignment, whenever the expression is false — i.e., `x<100` fails, and either `x>200` fails or `x!=y` fails.

3.4 Translating if-, if-else-, and while-Statements

3.4.1 The Grammar

$$S \rightarrow \text{if}(B) S_1 \mid \text{if}(B) S_1 \text{ else } S_2 \mid \text{while}(B) S_1$$

Nonterminal B represents a Boolean expression (as above) and nonterminal S represents a statement.

3.4.2 The Key Inherited Attributes: $B.true$, $B.false$, $S.next$

To generate correct jump targets, two labels are associated with every Boolean expression B :

- $B.true$ — the label to which control flows if B evaluates to true.
- $B.false$ — the label to which control flows if B evaluates to false.

And one inherited attribute is associated with every statement S :

- $S.next$ — the label of the instruction that should execute immediately *after* the code for S finishes (i.e., where control should fall through to once S is done). Using an explicit $S.next$ lets the translation avoid emitting a redundant “jump to the very next instruction” — a jump straight to L that is immediately followed by the label L can be replaced by simply falling through.

Intuition first. Picture each of `if`, `if-else`, and `while` as literally being built out of two known ingredients: the jumping code for the condition (which lands on either $B.true$ or $B.false$), and the code for the body statement(s). The translation schemes below just wire these two ingredients together with the right labels, in the right order, and add the loop-closing `goto` where needed.

- For `if (B) S1`: place $B.true$ right at the start of S_1 's code, and set $B.false$ equal to $S.next$ (whatever follows the whole `if`) — so a false condition skips straight past S_1 .

- For **if (B) S1 else S2**: place *B.true* at the start of S_1 , and *B.false* at the start of S_2 ; after S_1 's code, an explicit **goto S.next** is needed so control does not fall into S_2 's code; S_2 needs no such jump, since it is already immediately followed by *S.next*.
- For **while (B) S1**: a fresh label **begin** marks the top of the loop (where the condition is (re-)tested); *B.true* starts S_1 ; *B.false* is *S.next* (loop exit); and after S_1 's code, an unconditional **goto begin** re-tests the condition.

3.4.3 The Complete SDD (Formal Rules)

Production	Semantic Rules
$P \rightarrow S$	$S.next = newlabel(); \quad P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if}(B) S_1$	$B.true = newlabel(); \quad B.false = S_1.next = S.next; \quad S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if}(B) S_1 \text{ else } S_2$	$B.true = newlabel(); \quad B.false = newlabel(); \quad S_1.next = S_2.next = S.next; \quad S.code = B.code \parallel label(B.true) \parallel S_1.code \parallel gen('goto' S.next) \parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while}(B) S_1$	$begin = newlabel(); \quad B.true = newlabel(); \quad B.false = S.next; \quad S_1.next = begin; \quad S.code = label(begin) \parallel B.code \parallel label(B.true) \parallel S_1.code \parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel(); \quad S_2.next = S.next; \quad S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

Here `newlabel()` creates a brand-new label each time it is invoked, and `label(L)` attaches label *L* to the *next* three-address instruction to be generated. `assign` stands for an ordinary (non-control-flow) assignment statement, whose translation follows the rules already given in Part 1/Part 2.

Explanation, production by production.

- $P \rightarrow S$: the whole program is one statement; a final label *S.next* is created for “the instruction after everything,” and attached at the very end.
- $S \rightarrow \text{if}(B)S_1$: label *B.true* marks the first instruction of S_1 ; setting *B.false* = *S.next* means a false condition makes control skip S_1 entirely and fall through to whatever follows the **if**.
- $S \rightarrow \text{if}(B)S_1 \text{ else } S_2$: control flows out of *B.code* to S_1 's code if *B* is true, and to S_2 's code if *B* is false. Since both S_1 and S_2 must reach the same following instruction (*S.next*), an explicit **goto S.next** is emitted after S_1 's code (to skip over S_2); no such jump is needed after S_2 's code, because *S₂.next* already equals *S.next*, so S_2 's code is already immediately followed by that label.
- $S \rightarrow \text{while}(B)S_1$: label **begin** is a **local variable** of this production (not an attribute) marking the point the condition is retested; *S₁.next* is set to **begin** so that falling off the end of the loop body naturally loops back to the test (via the explicit **goto begin** that follows *S₁.code*).

- $S \rightarrow S_1 S_2$: the label at which control passes from S_1 to S_2 is a fresh label ($S_1.next$), attached right before S_2 's code begins.

Caution (efficiency note). If implemented completely literally, this SDD can generate more labels than strictly necessary, and may attach more than one label to the same instruction — Section “Avoiding Redundant Gotos” below discusses one way to reduce this; the *backpatching* technique (a related, one-pass alternative not covered by the material in this note) is another.

3.4.4 The SDD for Boolean Expressions in a Control-Flow Context

These rules complement the statement rules above: a Boolean expression B is translated into instructions that evaluate it using **conditional and unconditional jumps to one of two labels**, $B.true$ or $B.false$ — never into a stored value.

Production	Semantic Rules
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true; \quad B_1.false =$ $newlabel(); \quad B_2.true = B.true; \quad B_2.false =$ $B.false;$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel(); \quad B_1.false =$ $B.false; \quad B_2.true = B.true; \quad B_2.false =$ $B.false;$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow !B_1$	$B_1.true = B.false; \quad B_1.false =$ $B.true; \quad B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code \parallel$ $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto' B.true)$ $\parallel gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$

Explanation of each rule:

1. $B_1 \parallel B_2$: if B_1 is true, the whole B is already true, so $B_1.true$ is simply $B.true$ directly. If B_1 is false, B_2 must still be evaluated, so $B_1.false$ is set to a fresh label marking the start of B_2 's code; B_2 's own true/false exits become B 's true/false exits, since whatever B_2 decides is the final answer once we reach it.
2. $B_1 \&\& B_2$: symmetric to $\parallel$ — if B_1 is false, the whole B is already false ($B_1.false = B.false$ directly); if B_1 is true, B_2 must be evaluated, so $B_1.true$ is a fresh label starting B_2 's code, and B_2 's true/false exits become B 's own.
3. $!B_1$: no code at all is generated for the negation itself — it is implemented purely by **swapping** the true and false exit labels passed down to B_1 .
4. $E_1 \text{ rel } E_2$: translates directly into one relational conditional jump to $B.true$ if the comparison holds, followed unconditionally by a jump to $B.false$ if it does not (i.e., if the conditional jump instruction does not fire, execution simply falls into the following unconditional **goto** $B.false$).
5. **true/false constants**: translate into a single unconditional jump to the corresponding exit label.

3.4.5 Exam Revision Questions

Q1. Define the attributes `B.true`, `B.false`, and `S.next`, and explain why they are needed.

Answer: `B.true` is the label to which control transfers when Boolean expression `B` evaluates to true; `B.false` is the label to which control transfers when `B` evaluates to false. `S.next` is the label of the instruction that should execute immediately after statement `S` completes. They are needed because three-address code must express control flow using explicit jumps and labels; these attributes let the semantic rules correctly wire together the jump targets of nested conditions and statements so that control always ends up in the right place, and let redundant “jump to the next instruction” `gotos` be identified and avoided where the next label already equals `S.next`.

Q2. Why does the SDD for `if (B) S1 else S2` insert an explicit `goto S.next` after `S1`’s code but not after `S2`’s code?

Answer: After the true branch `S1` executes, control must skip over the false branch `S2` and land on the common continuation point `S.next`; without an explicit jump, control would simply fall through into `S2`’s code, which is wrong. After `S2` executes, however, no such jump is needed because `S2.next` is set equal to `S.next`, so `S2`’s code is already immediately followed by the label `S.next` — falling through is already correct.

Q3. In the while-loop translation scheme, what is the purpose of the local variable `begin`, and how does the loop close?

Answer: `begin` is a fresh label marking the point where the loop’s Boolean condition `B` is (re-)evaluated; it is not an attribute of any grammar symbol, just a value local to this production’s semantic action. The loop body `S1`’s `next` attribute is set to `begin` (so falling off the end of the body returns to the test), and the code additionally emits an explicit `goto begin` right after the body’s code, guaranteeing the loop re-tests its condition every iteration.

Q4. Explain how the SDD rule for `B1 || B2` implements short-circuit evaluation without generating any code for the `||` operator itself.

Answer: The rule sets `B1.true = B.true` directly, so if `B1` turns out true, control jumps straight to `B`’s true exit without ever reaching `B2`’s code — `B2` is skipped entirely, which is exactly short-circuiting. If `B1` is false, `B1.false` is a fresh label placed right before `B2`’s code, so control simply falls into evaluating `B2`, whose own true/false exits are inherited as `B`’s exits. No instruction is emitted purely to represent the `||` operator — the operator’s meaning is captured entirely by how the true/false labels are wired between `B1` and `B2`.

Q5. How is `!B1` (logical NOT) translated, and why does it require no additional code?

Answer: `!B1` is translated by simply swapping the inherited true/false labels before passing them down to `B1`: `B1.true = B.false` and `B1.false = B.true`. Since `B1`’s own code already jumps to whichever label it is given for “true” and “false,” swapping the labels alone reverses the sense of the entire condition — no extra instruction is needed.

Q6. Give the semantic rule for `B -> E1 rel E2`, and explain the purpose of each generated instruction.

Answer: $B.code = E_1.code \parallel E_2.code \parallel gen('if' E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true) \parallel gen('goto' B.false).$ The first two pieces compute the operand values. The conditional jump transfers control to `B.true` if the relation holds. If it does not hold, execution simply falls through to the very next instruction, which is an unconditional jump to `B.false` — guaranteeing that exactly one of the two exits is always taken.

3.5 Worked Simulation — Translating an if-Statement with Mixed OR/AND

Given: translate the statement

```
if ( x < 100 || x > 200 && x != y ) x = 0;
```

using the SDD's given above. Recall `&&` binds tighter than `||`, so the condition parses as $B_1 \parallel B_2$, where B_1 is $x < 100$ and B_2 is $x > 200 \ \&\& \ x \neq y$ (itself $B_3 \ \&\& \ B_4$, with $B_3 = x > 200$ and $B_4 = x \neq y$).

Step-by-step label generation (labels are created in the order the semantic rules are evaluated, top-down):

Step	Rule Firing	New Label Created	Meaning
1	$P \rightarrow S$	$L_1 = S.next$	Instruction after the whole if -statement
2	$S \rightarrow \text{if}(B)S_1$	$L_2 = B.true$	Start of S_1 's code ($x = 0$)
3	$B \rightarrow B_1 \parallel B_2$	$L_3 = B_1.false$	Start of B_2 's code, reached only if B_1 is false
4	$B_2 \rightarrow B_3 \ \&\& \ B_4$	$L_4 = B_3.true$	Start of B_4 's code, reached only if B_3 is true

Resulting inherited labels: $B_1.true = L_2$, $B_1.false = L_3$; $B_3.true = L_4$, $B_3.false = B_2.false = B.false = S_1.next = S.next = L_1$; $B_4.true = B_2.true = B.true = L_2$, $B_4.false = L_1$.

Generated code, piece by piece:

Component	Code
$B_1.code \ (x < 100)$	if x < 100 goto L2 then goto L3
$label(L_3)$	L3:
$B_3.code \ (x > 200)$	if x > 200 goto L4 then goto L1
$label(L_4)$	L4:
$B_4.code \ (x \neq y)$	if x != y goto L2 then goto L1
$label(L_2), S_1.code$	L2: x = 0
$label(L_1)$	L1:

Final three-address code:

```
if x < 100 goto L2
goto L3
L3: if x > 200 goto L4
    goto L1
L4: if x != y goto L2
    goto L1
```

```
L2: x = 0
L1:
```

Why this is correct. Control reaches L2 (and executes `x=0`) exactly when `x<100` is true, **or** when both `x>200` and `x!=y` are true — precisely the meaning of `x<100 || (x>200 && x!=y)`. Any other combination of outcomes falls through to L1, skipping the assignment.

An efficiency observation. This translation contains **three redundant instructions**: `goto L3` is immediately followed by the label L3, so it can simply be deleted (falling through achieves the same effect); similarly, the two `goto L1` instructions can be eliminated by replacing the preceding `if` instructions with `ifFalse` instructions instead. This observation motivates the fall-through technique described next.

3.6 Avoiding Redundant Gotos: The Fall-Through Technique

3.6.1 The Idea

An instruction sequence like

```
if x > 200 goto L4
goto L1
L4: ...
```

can be replaced by the single instruction

```
ifFalse x > 200 goto L1
L4: ...
```

The `ifFalse` instruction takes advantage of the *natural fall-through* from one instruction to the next: if `x > 200` is false, control simply “falls through” to whatever comes next — there is no need for an explicit `goto` at all.

In the code layouts for `if`- and `while`-statements, the code for the body statement S_1 **always** immediately follows the code for B . This means we can use a special label value, `fall` (meaning “do not generate a jump here — just fall through”), for $B.true$ in these situations.

3.6.2 Revised Rules Using `fall`

For $S \rightarrow \text{if}(B)S_1$: set $B.true = fall$, and $B.false = S_1.next = S.next$ (unchanged). Since S_1 ’s code already immediately follows B ’s code, no jump is needed for the true case — falling through *is* the true case.

For $B \rightarrow E_1 \text{ rel } E_2$, letting $test = E_1.addr \text{ rel.op } E_2.addr$:

- If **both** $B.true \neq fall$ and $B.false \neq fall$ (explicit labels on both sides): generate `if test goto B.true` followed by `goto B.false` (the original, non-optimized style).
- If **only** $B.true \neq fall$: generate just `if test goto B.true` (falling through covers the false case).
- If **only** $B.false \neq fall$: generate just `ifFalse test goto B.false` (falling through covers the true case).
- If **both** are `fall`: no jump instruction is generated at all.

For $B \rightarrow B_1 \parallel B_2$: $B_1.false$ is always **fall** (control must fall through from B_1 into B_2 's code when B_1 is false, exactly as before). But $B_1.true$ needs care: if $B.true$ is *already* an explicit label, that label is reused directly for $B_1.true$; but if $B.true$ is itself **fall**, then a **fresh label must be created** for $B_1.true$ anyway, because when B_1 is true, control must **jump over** B_2 's code to reach the instruction after B — it cannot simply “fall through,” since B_2 's code sits physically in the way.

3.6.3 Worked Example — Re-translating with fall-through

Using these new rules on the **same** statement as before, `if (x<100 || x>200 && x!=y) x=0;;`

- $S \rightarrow \text{if}(B)S_1$ now sets $B.true = \text{fall}$ (instead of a new label), and $B.false = S.next = L_1$ (as before, a genuine label is still needed since $x = 0$ must be *skippable*).
- For $B \rightarrow B_1 \parallel B_2$: since $B.true = \text{fall}$, a **new** label L_2 must still be created for $B_1.true$ (because if B_1 is true, control must jump over B_2 's code to reach `x=0`). $B_1.false = \text{fall}$ (falls through into B_2 's code). $B_2.true = B.true = \text{fall}$, $B_2.false = B.false = L_1$.
- For $B_2 = B_3 \&\& B_4$: analogous reasoning gives $B_3.true = \text{fall}$, $B_3.false = B_2.false = L_1$; $B_4.true = B_2.true = \text{fall}$, $B_4.false = L_1$.

Step-by-step generated code:

Component	$B.true / B.false$	Rule Used	Code
B_1 (<code>x<100</code>)	$true=L_2, false=\text{fall}$	only true explicit -> plain if	<code>if x < 100 goto L2</code>
B_3 (<code>x>200</code>)	$true=\text{fall}, false=L_1$	only false explicit -> ifFalse	<code>ifFalse x > 200 goto L1</code>
B_4 (<code>x!=y</code>)	$true=\text{fall}, false=L_1$	only false explicit -> ifFalse	<code>ifFalse x != y goto L1</code>
$\text{label}(L_2), S_1$	—	—	<code>L2: x = 0</code>
$\text{label}(L_1)$	—	—	<code>L1:</code>

Final, optimized three-address code:

```

if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2: x = 0
L1:

```

Comparison of instruction counts:

Version	Number of Instructions	Number of Unconditional Gotos
Original (explicit labels, Example above)	7	3
Fall-through (this section)	4	0

The fall-through version computes the *exact same logic* using fewer instructions and no unconditional jumps at all, by exploiting the fact that the code layout already places S_1 's code immediately after B 's code.

3.6.4 Exam Revision Questions

Q1. What does the special label value `fall` mean, and where is it used?

Answer: `fall` is a sentinel value meaning “do not generate a jump instruction for this exit — simply let control fall through to the instruction that follows in the code layout.” It is used for `B.true` in the `if`-statement rule (and similarly in `while`), since the code layout already places the body’s code immediately after the Boolean expression’s code, making an explicit jump for the true case unnecessary.

Q2. Give the `ifFalse` optimization rule for `B -> E1 rel E2` when only `B.false` is an explicit label (and `B.true` is `fall`).

Answer: Generate a single instruction, `ifFalse test goto B.false`, where `test` is `E1.addr rel.op E2.addr`. If the relation is false, control jumps to `B.false`; if the relation is true, control simply falls through to the next instruction (which is where `B.true`’s code — meaning `fall` — would have been reached anyway).

Q3. In the fall-through rule for `B1 || B2`, why must a genuine (non-fall) label sometimes be created for `B1.true`, even though `B.true` itself is `fall`?

Answer: Because when `B1` is true, the *entire* expression `B` is already true and execution must skip over `B2`’s code (which is physically placed right after `B1`’s code) to reach the code that follows `B`. Simply falling through from `B1` would incorrectly execute `B2`’s code as well. So whenever `B.true = fall`, a fresh explicit label is created for `B1.true`, and an actual jump to that label is generated, followed by placing that label right after `B2`’s code.

Q4. Re-translate `if (x<100 || x>200 && x!=y) x=0`; using the fall-through technique, and count how many instructions are saved compared with the non-optimized version.

Answer: The fall-through translation is: `if x<100 goto L2 / ifFalse x>200 goto L1 / ifFalse x!=y goto L1 / L2: x=0 / L1: —` four instructions. The non-optimized version (Example above) used seven instructions, including three unconditional `gotos`. The fall-through version saves three instructions (all three of the previously-redundant `gotos`), producing shorter code with identical behavior.

3.7 Boolean Values and Jumping Code

So far, this section has focused entirely on using Boolean expressions to alter the flow of control. But, as noted at the start of this part, a Boolean expression may also need to be **evaluated for its value**, as in an assignment such as `x = true`; or `x = a < b`;

A clean way to handle both roles. Build a syntax tree for expressions (using one of the approaches from Part 1 — full two-pass tree construction, or one pass for statements with two passes for expressions), then provide **two separate code-generation methods** on an expression’s syntax-tree node:

- **jump** — generates jumping code at that node, exactly as described throughout this section (using the rules keyed to `B.true`/`B.false`).
- **rvalue** — generates code to compute the node’s value into a temporary, the way ordinary arithmetic expressions are handled.

A single grammar with one expression nonterminal E can then serve both roles:

$$\begin{aligned}
 S &\rightarrow \text{id} = E; \mid \text{if}(E) S \mid \text{while}(E) S \mid S S \\
 E &\rightarrow E \parallel E \mid E \&\& E \mid E \text{ rel } E \mid E + E \mid (E) \mid \text{id} \mid \text{true} \mid \text{false}
 \end{aligned}$$

- When E appears in $S \rightarrow \mathbf{while}(E)S_1$ (governing flow of control), method **jump** is called on E 's syntax-tree node, generating code exactly as in the earlier SDD (with a fresh label t for the start of S_1 's code, and $S.next$ as the false exit).
- When E appears in $S \rightarrow \mathbf{id} = E$; (denoting a value), method **rvalue** is called instead. If E is arithmetic (e.g. $E_1 + E_2$), **rvalue** behaves exactly like ordinary expression translation (Part 1/2). If E has the form $E_1 \ \&\& \ E_2$ (or $||$), **rvalue** first generates *jumping* code for E (via the same mechanism as **jump**), and then, at the true and false exits of that jumping code, assigns **true** and **false** respectively into a new temporary — converting a control-flow result back into a stored value.

3.7.1 Worked Example — Assigning the Value of a Boolean Expression

Translate: $x = a < b \ \&\& \ c < d$;

The right-hand side, $a < b \ \&\& \ c < d$, is evaluated for its *value* (it is the right side of an assignment), so **rvalue** is invoked. **rvalue** first generates jumping code for the $\&\&$ expression (exactly following the SDD rules), sending the true exit to “assign true” and the false exit to “assign false”:

Step	Purpose	Generated Code
1	Test first conjunct; if false, skip to the false-assignment	<code>ifFalse a < b goto L1</code>
2	Test second conjunct; if false, skip to the false-assignment	<code>ifFalse c < d goto L1</code>
3	Both conjuncts true — assign true	<code>t = true</code>
4	Skip over the false-assignment	<code>goto L2</code>
5	False case — assign false	<code>L1: t = false</code>
6	Merge point — assign the computed Boolean value to x	<code>L2: x = t</code>

Final three-address code:

```

ifFalse a < b goto L1
ifFalse c < d goto L1
t = true
goto L2
L1: t = false
L2: x = t

```

Why this is correct. The code first behaves exactly like jumping code for $a < b \ \&\& \ c < d$ (using fall-through **ifFalse** tests, since only the false exit needs an explicit label here). If both tests pass, execution falls through to `t = true` and then jumps past the **false**-assignment. If either test fails, control jumps directly to L1 and assigns `t = false`. Either way, execution merges at L2, where the now-materialized Boolean value `t` is copied into `x` — turning the control-flow result back into an ordinary stored value.

3.7.2 Exam Revision Questions

Q1. Why can a single Boolean/arithmetic expression need two different translation strategies?

Answer: The same expression might be used either to alter control flow (as an `if/while` condition, where only the destination of control matters) or to compute a stored value (as the right side of an assignment, where an actual `true/false` result must end up in memory). A single “one-size-fits-all” translation cannot efficiently serve both needs, so compilers typically provide two separate code-generation procedures — one for jumping code, one for value code — and choose between them based on the syntactic context in which the expression appears.

Q2. Describe, in words, how the `rvalue` method translates a Boolean expression built from `&&/||` into a stored value.

Answer: It first generates ordinary jumping code for the expression (exactly as though the expression were controlling flow), but instead of that jumping code guarding a real statement, its true exit is followed by an instruction that assigns `true` into a new temporary, and its false exit is followed by an instruction that assigns `false` into the same temporary; an unconditional jump after the true-case assignment skips over the false-case assignment, and both paths merge at a final label where the temporary (now holding the Boolean’s actual value) can be used or copied into the target variable.

Q3. Translate `x = p > q`; (a single relational comparison assigned to a variable) using the value-computing (`rvalue`) strategy, showing all instructions.

Answer:

```
ifFalse p > q goto L1
t = true
goto L2
L1: t = false
L2: x = t
```

The relational test is evaluated once; if it holds, execution falls through to set `t = true` and skip the false-branch; otherwise it jumps to set `t = false`. Both paths converge at L2, where `t`’s now-concrete Boolean value is copied into `x`.

3.8 Full Simulation — A while-Loop with an OR Condition Followed by a Statement

This simulation (adapted from in-class practice) combines **everything** in this part — sequencing ($S_1 S_2$), a `while`-loop, an `||` condition, relational tests, and an assignment — into a single translation, giving the most complete practice exercise for exam preparation.

Given: translate the program fragment

```
while (a + b > 2 || a != 3) a = b + 3;
y = 3;
```

Parsing this fragment: the whole program is $S \rightarrow S_1 S_2$, where S_1 is the `while`-statement and S_2 is `y = 3`; S_1 itself is `while(B)S3`, where S_3 is `a = b + 3`; and B is $B_1 || B_2$, with $B_1 = a+b > 2$ and $B_2 = a != 3$.

3.8.1 Step 1 — Determine the Label-Generation Order

Labels are created in the order the semantic rules fire, working outside-in (an inherited attribute must be computed at a node before it can be passed down into that node’s children):

Order	Rule Firing	Label Created	What It Marks
1	$P \rightarrow S$	$L_1 = S.next$	Instruction after the entire fragment (the very end)
2	$S \rightarrow S_1 S_2$	$L_2 = S_1.next$	Instruction after the while -loop, i.e. the start of y=3 ;
3	$S_1 \rightarrow \mathbf{while}(B)S_3$	$L_3 = begin$	Top of the loop, where B is (re-)tested
4	$S_1 \rightarrow \mathbf{while}(B)S_3$	$L_4 = B.true$	Start of the loop body a = b+3 ;
5	$B \rightarrow B_1 \parallel B_2$	$L_5 = B_1.false$	Start of B_2 's code (a != 3), reached only if B_1 is false

3.8.2 Step 2 — Propagate the Inherited Attributes

Using the rules from the SDD:

- $S_1.next = L_2$ (from $S \rightarrow S_1 S_2$); since S_1 is a **while**-loop, its “exit” attribute is $B.false = S_1.next = L_2$, and $S_3.next = begin = L_3$.
- $B.true = L_4$ (fresh label from the **while** rule).
- $B_1.true = B.true = L_4$; $B_1.false = L_5$ (fresh, from the $\parallel$ rule).
- $B_2.true = B.true = L_4$; $B_2.false = B.false = L_2$.
- $S_2.next = S.next = L_1$.

3.8.3 Step 3 — Generate Code for Each Component

Component	Expression	Generated Code
$label(begin)$	—	L3:
$B_1.code(a+b > 2)$	compute a+b into t_1 , then test	t1 = a + b if t1 > 2 goto L4 goto L5
$label(B_1.false)$	—	L5:
$B_2.code(a \neq 3)$	test directly (no temporary needed — both operands are simple)	if a != 3 goto L4 goto L2
$label(B.true), S_3.code(a = b+3;)$	compute b+3 into t_2 , then assign	L4: t2 = b + 3 a = t2
loop-closing jump	—	goto L3
$label(S_1.next), S_2.code(y = 3;)$	—	L2: y = 3
$label(S.next)$	—	L1:

3.8.4 Step 4 — Final Generated Three-Address Code

```

L3: t1 = a + b
    if t1 > 2 goto L4
    goto L5
L5: if a != 3 goto L4
    goto L2
L4: t2 = b + 3
    a = t2
    goto L3
L2: y = 3
L1:

```

3.8.5 Verifying the Simulation

- **Loop test:** every iteration begins at L3, computes $a+b$ fresh, and tests $t1 > 2$. If true, control jumps directly to L4 (the loop body). If false, control falls through to L5 and tests the second disjunct $a \neq 3$; if *that* is true, control still reaches L4 (correctly implementing OR — either condition suffices); if both are false, control falls through to `goto L2`, which **exits the loop** and jumps straight to $y = 3$;
- **Loop body:** L4 computes $b+3$ into $t2$, assigns it to a , and then unconditionally jumps back to L3 to re-test the condition — the defining behavior of a **while-loop** (test-before-body, repeat).
- **After the loop:** label L2 is reached only when the condition is false, and it is exactly where $y = 3$ ’s code begins — confirming that $S_1.next$ (the label after the whole loop) was wired correctly to the statement that follows it in the source.

Why the labels appear “out of numerical order” in the final code (L3 first, then L5, then L4, then L2, then L1): labels are numbered in the order `newlabel()` is *called* during semantic-rule evaluation — which follows the grammar’s inherited-attribute dependencies (outermost $S.next$ first, then progressively inner labels) — **not** in the order the labeled instructions eventually appear in the output code. This is completely normal and is exactly what happens in the textbook’s own if-statement example (Example above): the generated code is always correct regardless of the numeric order in which the labels happen to have been created.

3.8.6 Exam Revision Questions

Q1. In the simulation `while (a+b>2 || a!=3) a=b+3; y=3;`, which label marks the top of the loop, and which marks the first instruction after the loop exits?

Answer: L3 (created as the local variable `begin` in the `while` rule) marks the top of the loop, where the condition is retested every iteration. L2 (created as $S_1.next$ from the $S \rightarrow S_1 S_2$ rule) marks the first instruction after the loop — the start of `y=3`; — and is exactly the label that `B.false` is wired to, so a false condition exits the loop straight into it.

Q2. Why does testing $a+b > 2$ require a temporary ($t1$), while testing $a \neq 3$ does not?

Answer: $a+b$ is a compound arithmetic expression with its own operator, so its value must first be computed into a temporary ($t_1 = a + b$) before it can be compared against 2. $a \neq 3$, on the other hand, compares two simple addresses (a and the constant 3) directly, so the relational instruction can reference them without any extra computation.

Q3. Explain why control falls through from B1’s code to L5’s code (the test for B2) when $a+b>2$ is false, rather than jumping to L2 (loop exit) directly.

Answer: Because the condition is $B1 \parallel B2$; when $B1$ is false, the overall truth of B is not yet determined — it still depends on $B2$. The semantic rule for $B1 \parallel B2$ therefore sets $B1.false$ to a fresh label (L_5) that marks the start of $B2$'s code, so evaluation continues with the second disjunct rather than exiting the loop prematurely. Only if *both* $B1$ and $B2$ turn out false does control reach the loop-exit jump (`goto L2`).

Q4. What three-address instruction guarantees that the while-loop actually repeats, and where is it placed?

Answer: The unconditional jump `goto L3` (i.e., `goto begin`), placed immediately after the loop body's code (`a = t2`). It sends control back to the top of the loop so the condition B is retested before deciding whether to execute the body again.

3.9 Practice Problems

3.9.1 Problem 1 (Fall 2025, Q6)

Draw the annotated parse tree and generate the final optimized three-address code produced by the SDT (using the fall-through technique, special label `fall`), for the string:

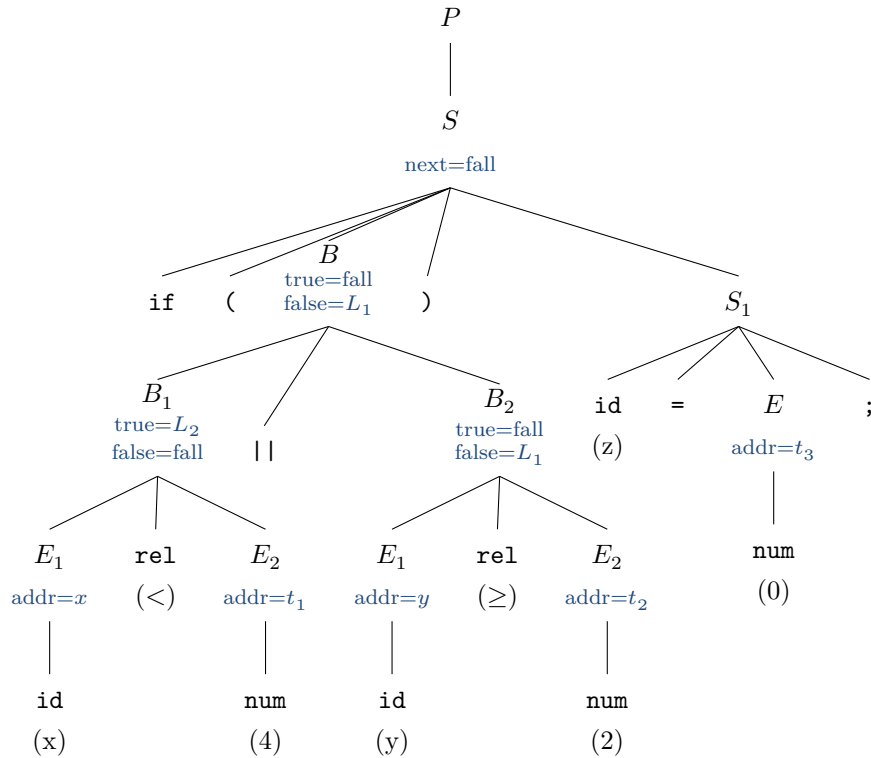
```
if ( x < 4 || y >= 2 ) z = 0;
```

Here $\parallel$ denotes string concatenation of generated code, and $\text{not}(\text{rel.op})$ denotes the algebraic negation of a relational operator, e.g. $\text{not}(\geq) = <$.

Solution. The SDT for this problem:

Production	Semantic Rules (as given in the exam)
$P \rightarrow S$	$S.next = \text{'fall'}$; $P.code = S.code$
$S \rightarrow \text{id} = E$;	$S.code = E.code \parallel$ $gen(\text{top.get}(\text{id.lexeme}) ' = ' E.addr)$
$S \rightarrow \text{if}(B)S_1$	$B.true = \text{'fall'}$; $S_1.next = S.next$; if $S.next = \text{'fall'}$: $B.false = \text{newLabel}()$; $S.code = B.code \parallel$ $S_1.code \parallel gen(B.false " : ")$ else: $B.false = S.next$; $S.code = B.code \parallel$ $S_1.code$
$B \rightarrow B_1 \parallel B_2$	if $B.true = \text{'fall'}$: $B_1.true = \text{newLabel}()$; $B_1.false =$ 'fall' ; $B_2.true = \text{'fall'}$; $B_2.false = B.false$; $B.code = B_1.code \parallel B_2.code \parallel$ $gen(B_1.true " : ")$ else: $B_1.true = B.true$; $B_1.false = \text{'fall'}$; $B_2.true =$ $B.true$; $B_2.false = B.false$; $B.code = B_1.code$ $\parallel B_2.code$

Production	Semantic Rules (as given in the exam)
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code;$ if $B.true = \text{'fall'}$: $B.code = B.code \parallel$ $gen("if" E_1.addr \text{ not}(\text{rel.op}) E_2.addr "goto" B.false)$ else if $B.false = \text{'fall'}$: $B.code = B.code \parallel$ $gen("if" E_1.addr \text{ rel.op } E_2.addr "goto" B.true)$ else: $B.code = B.code \parallel$ $gen("if" \dots "goto" B.true) \parallel$ $gen("goto" B.false)$
$E \rightarrow \text{id}$	$E.addr = top.get(\text{id.lexeme}); E.code = ""$
$E \rightarrow \text{num}$	$E.addr = newTemp(); E.code =$ $gen(E.addr ' = ' \text{ num.val})$

Annotated Parse Tree:

Step-by-step trace (the whole program is a single if-statement, so $S.next = \text{'fall'}$ from the very start; the condition is $B_1 \parallel B_2$ with $B_1 = x < 4$, $B_2 = y \geq 2$):

Step	Rule Firing	Result
1	$P \rightarrow S$	$S.next = \text{'fall'}$
2	$S \rightarrow \text{if}(B)S_1$	$B.true = \text{'fall'}$; $S_1.next = S.next = \text{'fall'}$; since $S.next = \text{'fall'}$, a genuine label is created: $B.false = newLabel() = L_1$

Step	Rule Firing	Result
3	$B \rightarrow B_1 \parallel B_2$, with $B.true = \text{'fall'}$	$B_1.true = newLabel() = L_2$; $B_1.false = \text{'fall'}$; $B_2.true = \text{'fall'}$; $B_2.false = B.false = L_1$
4	$B_1 \rightarrow E_1 \text{ rel } E_2$ ($x < 4$), $B_1.false = \text{'fall'}$	$E_1.addr = x$; $E_2.addr = newTemp() = t_1$, $gen(t_1 ' = ' 4)$; since $B_1.false = \text{'fall'}$, generate if $x < t_1$ goto L2
5	$B_2 \rightarrow E_1 \text{ rel } E_2$ ($y \geq 2$), $B_2.true = \text{'fall'}$	$E_1.addr = y$; $E_2.addr = newTemp() = t_2$, $gen(t_2 ' = ' 2)$; since $B_2.true = \text{'fall'}$, generate if $y \text{ not}(\geq) t_2$ goto L1, i.e. if $y < t_2$ goto L1
6	$S_1 \rightarrow \text{id} = E$; ($z=0$)	$E.addr = newTemp() = t_3$, $gen(t_3 ' = ' 0)$; $gen(z ' = ' t_3)$

Final three-address code:

```

t1 = 4
if x < t1 goto L2
t2 = 2
if y < t2 goto L1
L2: t3 = 0
    z = t3
L1:

```

3.9.2 Problem 2 (Spring 2026, Q1)

When we are generating three-address code for control-flow statements, why do we calculate labels as inherited attributes?

Solution. A label such as $B.true$, $B.false$, or $S.next$ tells a subtree **where to send control once it finishes** — and that destination is decided entirely by the *surrounding* context (the parent statement or the sibling that follows), not by anything inside the subtree itself. For example, in $S \rightarrow \text{if}(B)S_1$, the label $B.false$ is set to $S.next$ — the instruction after the *whole* if-statement — a piece of information that only the parent node (S) knows, and which must be pushed *down* into B before B 's own code can be generated. This is precisely the definition of an inherited attribute: a value computed at a node (or received from its parent) and passed down to its children, as opposed to a synthesized attribute (like $E.addr$ or $B.code$), which is built bottom-up from a node's children and passed up to its parent. Labels cannot be synthesized bottom-up because a subtree, taken in isolation, has no way to know what instruction should follow it in the final program — only the surrounding statement structure determines that — so they must flow top-down, i.e., be inherited.

3.9.3 Problem 3 (Spring 2026, Q5)

(a) The following CFG and SDD are for translating composite Boolean conditions involving the AND connective. Find any error in the SDD and write the correct SDD:

$$B \rightarrow B_1 \ \&\& \ B_2 \quad \{ B_1.true = B.true; B_1.false = newLabel(); B_2.true = B.true; B_2.false = B.false; \\ B.code = B_1.code \parallel\parallel gen(B_1.false \ " : ") \parallel\parallel B_2.code; \}$$

Solution. The correct rule is $B_1.true = newlabel(); B_1.false = B.false; B_2.true = B.true; B_2.false = B.false$. The given SDT has **swapped the roles of $B_1.true$ and $B_1.false$** : it sets $B_1.true = B.true$ (a value that should only be assigned when B_1 evaluating true is *already enough* to make the whole expression true) and creates the new label for $B_1.false$ instead — this is the rule for OR, not AND. With the given (incorrect) rule, as soon as B_1 is true, control jumps straight to $B.true$ **without ever testing** B_2 , i.e. the expression is treated as true whenever *either* operand is true. For a correct AND, if B_1 is false the whole expression must *immediately* be false (short-circuit to $B.false$); only if B_1 is true does B_2 still need to be evaluated.

Corrected SDD:

$$B \rightarrow B_1 \ \&\& \ B_2 \quad \{ B_1.true = newLabel(); B_1.false = B.false; B_2.true = B.true; B_2.false = B.false; \\ B.code = B_1.code \parallel\parallel gen(B_1.true \ " : ") \parallel\parallel B_2.code; \}$$

$B_1.false$ is set directly to $B.false$ (if B_1 fails, the whole AND fails immediately — no new label needed), while a **fresh** label is created for $B_1.true$ (marks where B_2 's code begins, reached only if B_1 succeeds); B_2 's own true/false exits are simply inherited as B 's own, since B_2 's outcome is the final answer once reached.

(b) Why does short-circuit evaluation improve the generated three-address code for Boolean expressions? Give an example.

Solution. Short-circuit evaluation avoids emitting (and, at run time, executing) code for a sub-expression whose value cannot change the final result once the other operand's value is known — for $B_1 \parallel\parallel B_2$, once B_1 is true the whole expression is already true, and for $B_1 \ \&\& \ B_2$, once B_1 is false the whole expression is already false. The SDD simply routes B_1 's already-true (or already-false) exit directly to B 's own exit, and B_2 's code is generated but only *reached* along the path where it is actually needed — no explicit runtime check or extra instruction is required to skip it. Short-circuiting is sometimes semantically necessary, not just an optimization: in `if (x != 0 && 10/x > 2) ...`, evaluating $10/x$ when $x==0$ would cause a division-by-zero error; short-circuit evaluation guarantees $10/x$ is only reached once $x \neq 0$ has already been confirmed true.

3.9.4 Problem 4 (Spring 2025, Q4)

Consider the following SDT. Draw the annotated parse tree and evaluate it for the string:

```
while (x < y) z = u + v - x; y = z * u;
```

Solution. This SDT uses a **single shared global counter** p , incremented at specific embedded actions scattered throughout the grammar, to number *both* labels and temporaries — `newlabel()` returns `"L"+p` and `newtemp()` returns `"t"+p`, always using whatever value p currently holds:

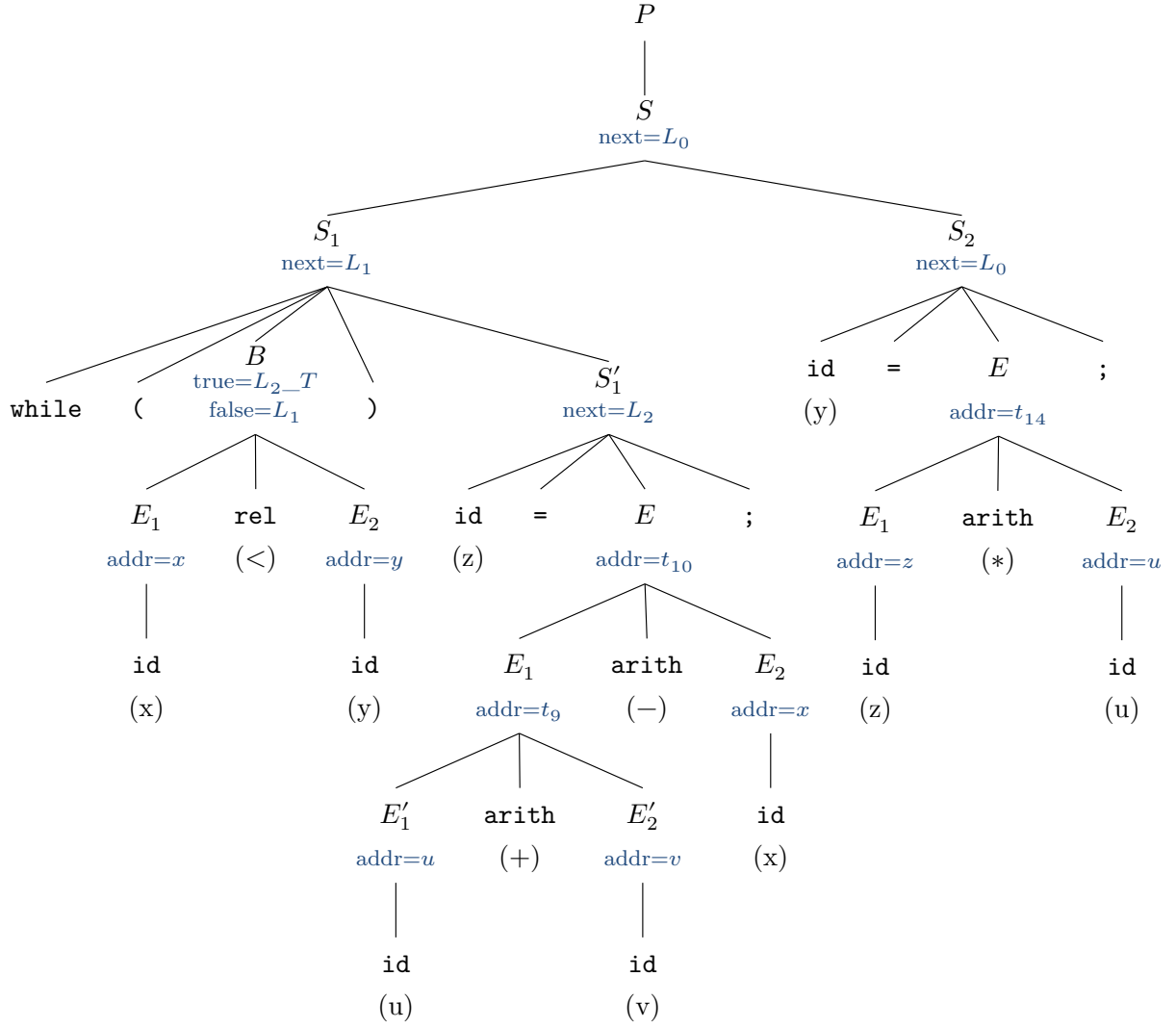
Production	Semantic Rules (as given in the exam)
$P \rightarrow S$	$\{p = 0; S.next = newlabel();\}$ then parse S ; $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{Assign}$	$\{p++;\}$ then parse Assign ; $S.code = \text{Assign}.code$
$\text{Assign} \rightarrow \text{id} = E;$	$\text{Assign}.code = E.code \parallel$ $gen(\text{id.lexeme} \text{ ' = ' } E.addr)$
$S \rightarrow \text{while}(B)S_1$	$\{p++; begin = newlabel(); B.true =$ $newlabel() + \text{"_T"}; B.false = S.next;\}$, then B , then $\{p++; S_1.next = begin;\}$, then S_1 ; $S.code = label(begin) \parallel B.code \parallel label(B.true)$ $\parallel S_1.code \parallel gen(\text{'goto' } begin)$
$S \rightarrow S_1 S_2$	$\{p++; S_1.next = newlabel();\}$, then S_1 , then $\{p++; S_2.next = S.next;\}$, then S_2 ; $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$
$B \rightarrow E_1 \text{ rel } E_2$	$\{p++;\}$, then E_1 , then $\{p++;\}$, then E_2 ; $B.code = E_1.code \parallel E_2.code \parallel$ $gen(\text{'if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$ $\parallel gen(\text{'goto' } B.false)$
$E \rightarrow E_1 \text{ arith } E_2$	$\{p++;\}$, then E_1 , then $\{p++;\}$, then E_2 ; $E.addr = newtemp(); E.code = E_1.code \parallel$ $E_2.code \parallel$ $gen(E.addr \text{ ' = ' } E_1.addr \text{ arith.op } E_2.addr)$
$E \rightarrow \text{id}$	$E.addr = \text{id.lexeme}; E.code = \text{" "}$
$E \rightarrow \text{num}$	$E.addr = \text{num.value}; E.code = \text{" "}$

(Only the productions actually used by this input string are shown; the **if/if-else** productions are omitted as they are not needed here.)

Because `newlabel()` and `newtemp()` both read the *same* counter `p`, and `p` is incremented at scattered $\{p++;\}$ points across many different productions, every action firing must be traced strictly in the order the parser reaches it (left to right, outermost-first for actions that appear before a nonterminal is expanded), recording `p`'s value every time it changes.

The whole fragment is $S \rightarrow S_1 S_2$, with $S_1 = \text{while}(x < y) \ z = u + v - x$; and $S_2 = y = z * u$. S_1 is **while**(B) S'_1 with $B = x < y$ and $S'_1 = z = u + v - x$; (an **Assign**). S'_1 's expression $u + v - x$ parses left-associatively as $(u + v) - x$.

Annotated Parse Tree:

**Step-by-step firing trace:**

#	Action Fired	Effect	p After
1	$P \rightarrow S: p = 0$	—	0
2	$P \rightarrow S:$ $S.next = newlabel()$	$S.next = L_0$	0
3	$S \rightarrow S_1 S_2: p++$	—	1
4	$S \rightarrow S_1 S_2:$ $S_1.next = newlabel()$	$S_1.next = L_1$	1
5	$S_1 \rightarrow \mathbf{while}(B)S'_1:$ $p++$	—	2
6	$begin = newlabel()$	$begin = L_2$	2
7	$B.true =$ $newlabel() + \text{"_T"}$	$B.true = L_2_T$	2
8	$B.false = S_1.next$	$B.false = L_1$	2
9	$B \rightarrow E_1 \mathbf{rel} E_2 (x < y):$ $p++$	—	3
10	parse $E_1 = x$	$E_1.addr = x$	3

#	Action Fired	Effect	p After
11	$p++$	—	4
12	parse $E_2 = y$	$E_2.addr = y$	4
13	$B.code = gen('if\ x <$ $y\ goto\ L_2_T') \parallel$ $gen('goto\ L_1')$	—	4
14	back in while rule: $p++$	—	5
15	$S'_1.next = begin = L_2$	—	5
16	$S'_1 \rightarrow \mathbf{Assign}$ $(z=u+v-x); p++$	—	6
17	$E \rightarrow E_1 \mathbf{arith} E_2$ $((u+v)-x, \text{outer}):$ $p++$	—	7
18	$E_1 \rightarrow E'_1 \mathbf{arith} E'_2$ $(u+v, \text{inner}): p++$	—	8
19	parse $E'_1 = u$	$addr = u$	8
20	$p++$	—	9
21	parse $E'_2 = v$	$addr = v$	9
22	$E_1.addr = newtemp()$	$E_1.addr = t_9;$ $gen(t_9' = u + v)$	9
23	back in outer $E: p++$	—	10
24	parse $E_2 = x$	$addr = x$	10
25	$E.addr = newtemp()$	$E.addr = t_{10};$ $gen(t_{10} = t_9 - x)$	10
26	$\mathbf{Assign}.code =$ $gen(z = t_{10})$	—	10
27	back in $S \rightarrow S_1 S_2:$ $p++$	—	11
28	$S_2.next = S.next =$ L_0	—	11
29	$S_2 \rightarrow \mathbf{Assign}$ $(y=z*u); p++$	—	12
30	$E \rightarrow E_1 \mathbf{arith} E_2$ $(z*u): p++$	—	13
31	parse $E_1 = z$	$addr = z$	13
32	$p++$	—	14
33	parse $E_2 = u$	$addr = u$	14
34	$E.addr = newtemp()$	$E.addr = t_{14};$ $gen(t_{14} = z * u)$	14
35	$\mathbf{Assign}.code =$ $gen(y = t_{14})$	—	14

Assembling the code, following each production's code rule:

- $S'_1.code$ ($z=u+v-x$;) equals $t9 = u + v$ then $t10 = t9 - x$ then $z = t10$.
- $S_1.code$ (**while**) = $label(L_2) \parallel B.code \parallel label(L_2_T) \parallel S'_1.code \parallel gen('goto' L'_2)$.
- $S_2.code$ ($y=z*u$;) = $t14 = z * u$ then $y = t14$.
- $S.code = S_1.code \parallel label(L_1) \parallel S_2.code$.
- $P.code = S.code \parallel label(L_0)$.

Final three-address code:

```
L2:    if x < y goto L2_T
      goto L1
L2_T:  t9 = u + v
      t10 = t9 - x
      z = t10
      goto L2
L1:    t14 = z * u
      y = t14
L0:
```

3.9.5 Problem 5 (Q1)

Consider the following SDT. Draw the annotated parse tree and evaluate it for the string:

```
if (a < b) c = f; else p = q * r;
```

Solution. This SDT uses a **single shared global counter p** — `newlabel()` returns "L"+p and `newtemp()` returns "t"+p, always using whatever value p currently holds — together with the if-else production:

Production	Semantic Rules (as given in the exam)
$P \rightarrow S$	$\{p = 0; S.next = newlabel();\}$ then parse S ; $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{Assign}$	$\{p++;\}$ then parse Assign ; $S.code = \text{Assign}.code$
$\text{Assign} \rightarrow id = E;$	$\text{Assign}.code = E.code \parallel$ $gen(id.lexeme ' = ' E.addr)$
$S \rightarrow \text{if}(B)S_1 \text{ else } S_2$	$\{p++; B.true = newlabel() + _T; B.false =$ $newlabel() + _F;\}$, then B , then $\{p++; S_1.next = S.next;\}$, then S_1 , then else , then $\{p++; S_2.next = S.next;\}$, then S_2 ; $S.code = B.code \parallel label(B.true) \parallel S_1.code \parallel$ $gen('goto' S.next) \parallel label(B.false) \parallel S_2.code$
$B \rightarrow E_1 \text{ rel } E_2$	$\{p++;\}$, then E_1 , then $\{p++;\}$, then E_2 ; $B.code = E_1.code \parallel E_2.code \parallel$ $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto' B.true)$ $\parallel gen('goto' B.false)$

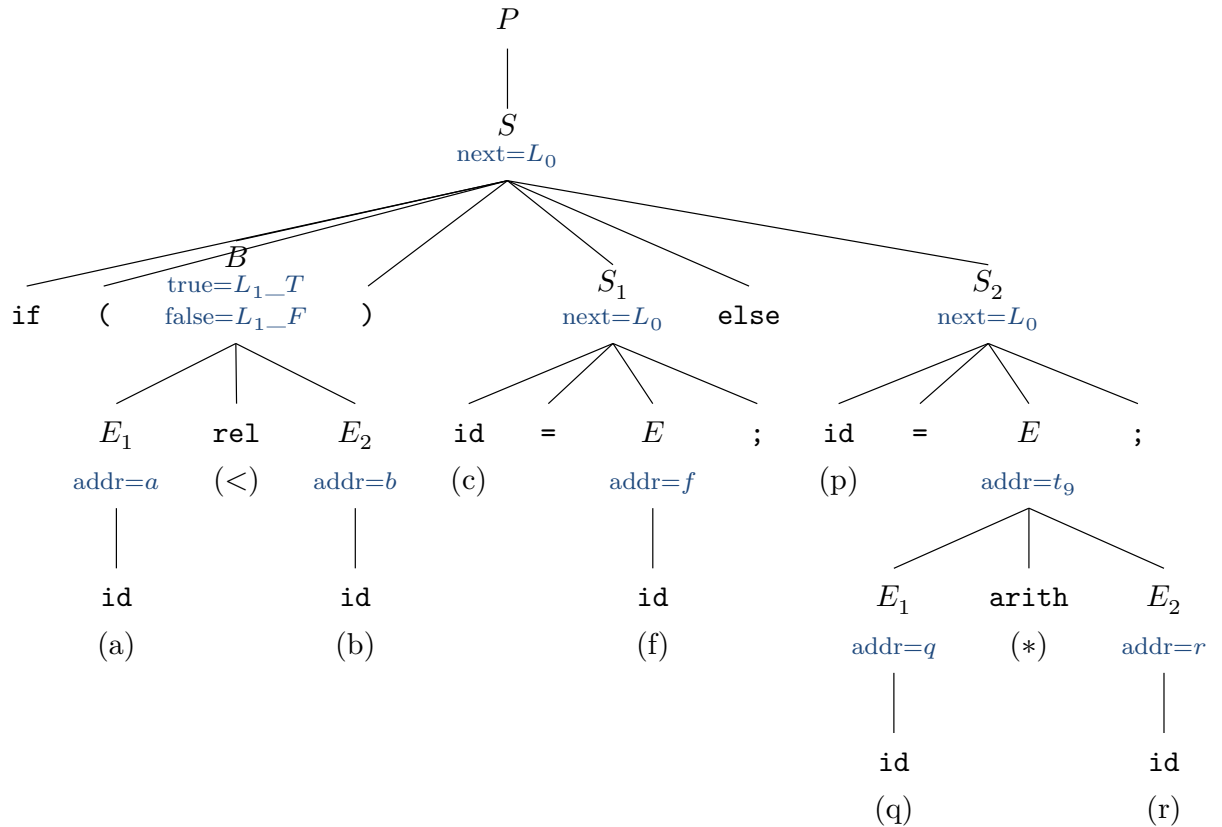
Production	Semantic Rules (as given in the exam)
$E \rightarrow E_1 \text{ arith } E_2$	$\{p++;\}$, then E_1 , then $\{p++;\}$, then E_2 ; $E.addr = \text{newtemp}(); E.code = E_1.code \parallel E_2.code \parallel$ $\text{gen}(E.addr \text{ '}' '=' E_1.addr \text{ arith.op } E_2.addr)$
$E \rightarrow \text{id}$	$E.addr = \text{id.lexeme}; E.code = ""$
$E \rightarrow \text{num}$	$E.addr = \text{num.value}; E.code = ""$

(Only the productions actually used by this input string are shown; $S \rightarrow \text{while}(B)S_1$ and $S \rightarrow S_1 S_2$ are omitted as they are not needed here.)

The **if-else** production fires **three** separate $\{p++;\}$ actions of its own (one before B , one before S_1 , one before S_2 right after the **else** keyword), on top of whatever the $S \rightarrow \text{Assign}$ production and the expressions inside S_1/S_2 each add, so every action must be traced strictly left to right, outside-in, recording p every time it changes. Note that the source program itself has a variable literally named p (in $p = q * r$), which is entirely unrelated to the SDT's own bookkeeping counter p — every unquoted p in the trace below is the counter, never the variable.

The whole fragment is a single $S \rightarrow \text{if}(B)S_1 \text{ else } S_2$, with $B = a < b$, $S_1 = c = f$; (an **Assign** with $E \rightarrow \text{id}$), and $S_2 = p = q * r$; (an **Assign** whose expression $q * r$ is $E \rightarrow E_1 \text{ arith } E_2$).

Annotated Parse Tree:



Step-by-step firing trace:

#	Action Fired	Effect	p After
1	$P \rightarrow S: p = 0$	—	0
2	$P \rightarrow S:$ $S.next = newlabel()$	$S.next = L_0$	0
3	$S \rightarrow \text{if}(B)S_1\text{else}S_2:$ $p++$	—	1
4	$B.true =$ $newlabel() + \text{"_T"}$	$B.true = L_{1_T}$	1
5	$B.false =$ $newlabel() + \text{"_F"}$	$B.false = L_{1_F}$	1
6	$B \rightarrow E_1 \text{rel } E_2 \text{ (a<b)}:$ $p++$	—	2
7	parse $E_1 = a$	$E_1.addr = a$	2
8	$p++$	—	3
9	parse $E_2 = b$	$E_2.addr = b$	3
10	$B.code = gen('if a <$ $b \text{ goto } L_{1_T'} \parallel$ $gen('goto L_{1_F}')$	—	3
11	back in if-else rule: $p++;$ $S_1.next = S.next$	$S_1.next = L_0$	4
12	$S_1 \rightarrow \text{Assign (c=f)}:$ $p++$	—	5
13	parse $E = f$	$E.addr = f$	5
14	Assign.code = $gen('c' ' = ' f);$ $S_1.code =$ Assign.code	—	5
15	back in if-else rule, after else : $p++;$ $S_2.next = S.next$	$S_2.next = L_0$	6
16	$S_2 \rightarrow \text{Assign}$ $(p=q*r): p++$	—	7
17	$E \rightarrow E_1 \text{arith } E_2$ $(q*r): p++$	—	8
18	parse $E_1 = q$	$E_1.addr = q$	8
19	$p++$	—	9
20	parse $E_2 = r$	$E_2.addr = r$	9
21	$E.addr = newtemp()$	$E.addr = t_9;$ $gen(t_9 ' = ' q * r)$	9

22

Assign.code = —
 $gen('p' = t_9);$
S₂.code =
Assign.code

9

Assembling the code, following each production's code rule:

- $S_1.code (c=f;) = c = f.$
- $S_2.code (p=q*r;) = t_9 = q * r \text{ then } p = t_9.$
- $S.code = B.code \parallel label(B.true) \parallel S_1.code \parallel gen('goto' S.next) \parallel label(B.false) \parallel S_2.code.$
- $P.code = S.code \parallel label(S.next).$

Final three-address code:

```
if a < b goto L1_T
goto L1_F
L1_T: c = f
      goto L0
L1_F: t9 = q * r
      p = t9
L0:
```